

Tangent-Space Methods for Nonlinear Control and Biomechanics

Volume IV: Human Motor Control
The Neural Architecture of Movement

Preface

Draft Notice. This volume is under active development. Chapter depth and validation vary; some sections remain preliminary. The biology and nonlinear-dynamics chapter includes worked examples and exercises with independently checked mechanics. That review does not certify every chapter, a neural mechanism, or a human performance model. Source and validation records distinguish the completed corrections from the remaining review.

Volumes 0–III developed the mathematical and physical foundations for understanding motion: differential geometry, nonlinear control, trajectory optimization, and biomechanics. This volume asks a fundamentally different question: *how does the brain actually control the body?*

The golfer coordinates a body, a flexible implement and changing contacts using actuators whose force capacity depends on activation and mechanical history. The relevant model dimension, movement duration, precision and feedback latency depend on the task and measurement; they cannot be reduced to one universal count or timing budget.

This volume investigates how parallel processing, recurrent feedback, prediction, learned policies and intrinsic mechanics can cooperate. Cortical histological layers are not sequential computational layers, and a synaptic delay divided into a movement duration does not establish network depth or prove one neural architecture. A low-dimensional or shallow controller can be a useful engineering construction without uniquely explaining human control.

The purpose is to connect explicit dynamical models with observations that could support or challenge them. For golf and humanoid manipulation alike, the questions concern available control authority, coupled stability, energy and task accuracy under stated physical and informational constraints.

Contents

Preface	i
Nomenclature	v
1 The Degrees-of-Freedom Problem	1
1.1 Bernstein’s Original Formulation	1
1.1.1 The Numbers Are Staggering	1
1.2 From Problem to Resource: Motor Abundance	2
1.3 The Uncontrolled Manifold Hypothesis	3
1.4 Bernstein’s Three Stages of Learning	5
2 The Curse of Dimensionality — And How Humans Solve It	7
2.1 Classical Curse of Dimensionality	7
2.2 Biological Solutions	7
2.2.1 1. Synergy-Based Compression	8
2.2.2 2. Hierarchical Decomposition	8
2.2.3 3. Passive Dynamics Exploitation	8
2.2.4 4. Internal Models and Prediction	8
2.2.5 5. Task-Space Control	8
3 Neural Architecture of Motor Control	10
3.1 Cortical Motor Areas	10
3.1.1 Primary Motor Cortex (M1)	11
3.1.2 Premotor Cortex (PMC) and Supplementary Motor Area (SMA)	11
3.2 Subcortical Motor Structures	11
3.2.1 Cerebellum: The Forward Model	11
3.2.2 Basal Ganglia: Action Selection	12
3.3 Spinal Cord: The Local Controller	12
4 Shallow-but-Wide: Why Brains Are Parallel, Not Deep	14
4.1 The Speed Constraint	14
4.1.1 The Depth Budget	14
4.2 The Motor Cortex: Six Layers, Not Sixty	15
4.3 Computational Implications	16
4.3.1 Complex Computations Through Simple Local Rules	16
4.3.2 Hierarchical Decomposition	18
4.3.3 Synergies as Dimensionality Reduction	19
4.4 The Shallow-Wide Hypothesis for Motor Control	21
5 Ideomotor Theory and the Predictive Brain	23
5.1 Historical Ideomotor Theory	23
5.2 The Predictive Processing Framework	24
5.2.1 Active Inference	24

5.3	Connection to Control Theory	25
6	Internal Models: Forward and Inverse	26
6.1	Forward Models	26
6.1.1	The MOSAIC Architecture	27
6.2	Inverse Models	28
6.3	Smith Predictor for Neural Delays	28
7	Passive Distributed Control	29
7.1	The Frans Bosch Framework	29
7.2	Passive Dynamic Walking	30
7.3	Passive Dynamics in the Golf Swing	31
7.4	Bernstein's Stage 3: Exploiting DOF	31
8	Biology and Nonlinear Dynamics	33
8.1	Introduction: Mechanics With a Biological State	33
8.1.1	Bernstein, Constraints and the Scope of the Claim	33
8.2	Muscle, Tendon and Activation Dynamics	34
8.2.1	A Consistent Hill-Type Model	34
8.2.2	Length and Velocity Conventions That Can Be Checked	35
8.2.3	Zero Excitation Retains the Activation History	35
8.2.4	Preflex Response and Tendon Energy	36
8.3	Attractors and Coordination Transitions	36
8.3.1	What Stability Does and Does Not Establish	36
8.3.2	A Cost Crossing Is Not a Dynamical Bifurcation	37
8.3.3	The Haken–Kelso–Bunz Model With Its Correct Transition	37
8.4	Synergies, Task Geometry and Control Authority	38
8.4.1	What a Low-Dimensional Fit Establishes	38
8.4.2	A Task Level Set and Its Tangent Space	39
8.4.3	Instantaneous Rank Is Not Controllability	40
8.5	Impedance and Co-Contraction	40
8.5.1	Force, Displacement and Velocity Ports	40
8.5.2	How Muscle Forces Contribute to Joint Stiffness	41
8.5.3	An Explicit Optimization Example	41
8.5.4	Mechanical Work and Metabolic Cost	42
8.6	Pre-Stress and Stability of Interacting Subsystems	42
8.6.1	What a Tensegrity Analogy Can Contribute	42
8.6.2	Why Stable Parts Can Form an Unstable Whole	43
8.7	Building and Testing a Coupled Movement Model	43
8.7.1	Choose the Model for the Question	43
8.7.2	Identification Requires More Than a Motion Fit	44
8.7.3	A Golf and Humanoid Experiment Ladder	44
8.7.4	What Would Count Against the Proposed Explanation?	45
8.8	Connection to the Series	45
9	Central Pattern Generators	47
9.1	CPGs in Biological Motor Control	47
9.2	The Matsuoka Oscillator	48
9.3	Phase Coupling and Inter-Limb Coordination	49

10 Motor Learning and Adaptation	50
10.1 Three Learning Systems	50
10.2 Visuomotor Adaptation	50
10.3 Savings and Interference	52
10.3.1 Savings	52
10.3.2 Interference and Dual-Rate Learning	52
10.4 Structural Learning and Meta-Learning	53
11 Computational Models of Motor Control	54
11.1 Optimal Feedback Control	54
11.1.1 The Minimum Intervention Principle	55
11.2 Signal-Dependent Noise	56
11.3 Active Inference (Friston Framework)	57
12 From Neural Architecture to Robot Control	58
12.1 Lessons From Biology for Robot Design	58
12.2 Cerebellar-Inspired Adaptive Control	59
12.3 CPG-Based Locomotion Controllers	59
12.4 Whole-Body Control Architectures	61
12.5 Companion Software: Practical Implementation	61

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Joint configuration vector (generalized coordinates).

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Joint velocity vector (generalized velocities).

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Joint acceleration vector.

$\mathbf{x} \in \mathbb{R}^{n_x}$ System state vector; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ for mechanical systems.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces/torques acting on the joints.

$t \in \mathbb{R}$ Time.

Inertia and Dynamics

$\mathbf{M}(\mathbf{q})$ Mass (inertia) matrix, symmetric positive definite.

$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ Coriolis and centrifugal force matrix.

$\mathbf{g}(\mathbf{q})$ Gravity vector.

$\mathbf{J}(\mathbf{q})$ Jacobian matrix relating joint velocities to task-space velocities ($\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$).

Control and Optimization

$\mathbf{A}_k = \frac{\partial f}{\partial \mathbf{x}}$ Local Jacobian matrix of the state dynamics.

$\mathbf{B}_k = \frac{\partial f}{\partial \mathbf{u}}$ Local Jacobian matrix of the control input.

$V(\mathbf{x})$ or $\mathcal{V}$ Value function or Lyapunov function.

$\mathbf{Q}$ State penalty matrix in LQR/DDP.

R Control penalty matrix in LQR/DDP.

K Feedback gain matrix ($\delta \mathbf{u} = -\mathbf{K}\delta \mathbf{x}$).

γ A trajectory or flow, mapping time and initial conditions to states.

Affine Control and Counterfactuals

$f(\mathbf{x})$ Drift vector field: complete autonomous evolution of the declared effective plant when $\mathbf{u} = \mathbf{0}$.

$G(\mathbf{x})$ Input (control) matrix field mapping control inputs to state derivatives.

$\dot{\mathbf{x}} = f(\mathbf{x}) + G(\mathbf{x})\mathbf{u}$ Control-affine system equations of motion.

$\mathbf{x}^{\text{ZTCF}}(t)$ Forward Zero-Torque Counterfactual trajectory under $\mathbf{u}(t) \equiv \mathbf{0}$.

$\mathbf{a}_{\text{ZVCF}}(\mathbf{q}, \mathbf{z})$ Instantaneous Zero-Velocity Counterfactual acceleration with velocity and declared control set to zero.

$\text{DCR}_{W, \mathcal{U}}(\mathbf{x})$ Drift-Control Ratio in a declared acceleration or task-projected space: $\frac{\|W \mathbf{a}_{\text{drift}}\|_2}{\sup_{\mathbf{u} \in \mathcal{U}} \|W B_a \mathbf{u}\|_2 + \varepsilon}$.

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

$\mathbf{F}_{\text{muscle}}$ Force generated by muscles.

$\mathbf{W}$ Muscle synergy matrix (weights).

$\mathbf{c}(t)$ Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

The Degrees-of-Freedom Problem

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The coordination of movement is the process of mastering redundant degrees of freedom of the moving organ, in other words its conversion to a controllable system.

— Nikolai Bernstein, 1967

1.1 Bernstein’s Original Formulation

In 1935, Nikolai Bernstein, a Soviet physiologist, made a deceptively simple observation: the number of degrees of freedom available for any motor task *vastly exceeds* the number required to define the task. This observation, formalized in his 1967 monograph [5], is the founding problem of motor control.

Definition 1.1. The Degrees-of-Freedom Problem

Let a motor task be defined by k task-space constraints (e.g., $k = 6$ for placing a hand at a specific position and orientation). Let the body have n joint degrees of freedom and $p > n$ muscles. The **degrees-of-freedom problem** is: how does the nervous system select a specific solution $(\mathbf{q}(t), \mathbf{u}(t)) \in \mathbb{R}^n \times \mathbb{R}^p$ from the infinite set of solutions consistent with the task constraints?

The **kinematic redundancy** is $n - k$ (extra joint configurations). The **muscle redundancy** is $p - n$ (extra muscles per joint). The **total redundancy** is $(n - k) + (p - n) = p - k$.

1.1.1 The Numbers Are Staggering

Consider the golf swing:

Quantity	Approximate Value
Skeletal DOF involved	~ 200
Muscles contributing	~ 400
Task constraints (clubhead at impact)	6
Kinematic redundancy	~ 194
Muscle redundancy	~ 200
Total redundancy	~ 394
Duration of downswing	~ 0.25 s
Neural processing delay	$\sim 0.1 - 0.2$ s

The nervous system must select from ~ 400 -dimensional muscle activation patterns to satisfy 6 constraints, while accounting for interaction torques, gravity, and neural delays—in *a quarter of a second*. This is the problem.

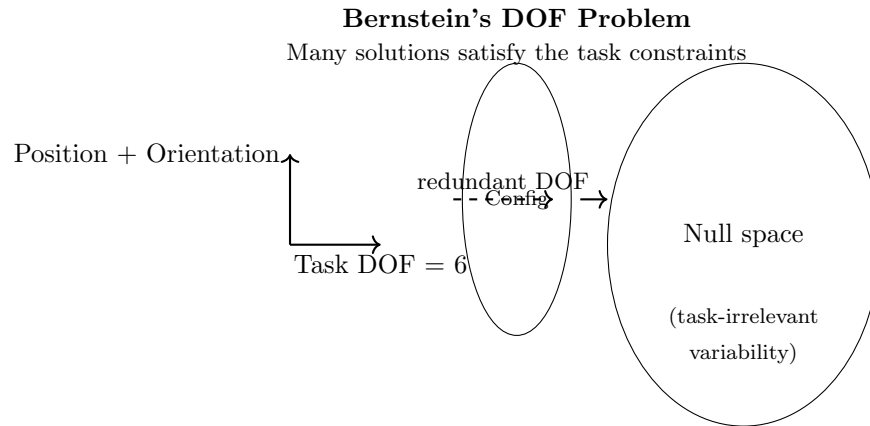


Figure 1.1: The degrees-of-freedom problem: redundancy in configuration space allows task-irrelevant variability in the null space of the task Jacobian.

1.2 From Problem to Resource: Motor Abundance

Modern motor control theory has reframed Bernstein's “problem” as an *opportunity*. Redundancy is not a curse but a resource:

Key Insight 1.1. Redundancy as Motor Abundance

The extra degrees of freedom provide:

1. **Flexibility:** the same task can be performed in many ways, allowing adaptation to constraints (fatigue, injury, obstacles)
2. **Robustness:** perturbations can be absorbed without task failure

3. **Exploration:** variability in execution allows the system to discover better solutions over time
4. **Error correction:** task-irrelevant DOF can compensate for errors in task-relevant DOF

We adopt the term **motor abundance** (Latash 2012) to emphasize that redundancy is a feature, not a bug.

1.3 The Uncontrolled Manifold Hypothesis

The uncontrolled manifold (UCM) hypothesis [35] provides a mathematical framework for understanding how the nervous system manages redundancy.

Definition 1.2. The Uncontrolled Manifold

Let $\mathbf{q} \in \mathbb{R}^n$ be the joint configuration and $\mathbf{y} = h(\mathbf{q}) \in \mathbb{R}^k$ be the task variable. The **uncontrolled manifold** at a nominal configuration $\mathbf{q}_0$ is the null space of the task Jacobian:

$$\text{UCM}(\mathbf{q}_0) = \{\delta\mathbf{q} \in \mathbb{R}^n : J_h(\mathbf{q}_0)\delta\mathbf{q} = \mathbf{0}\}, \quad (1.1)$$

where $J_h = \partial h / \partial \mathbf{q}$ is the task Jacobian. Variability within the UCM does not affect the task variable. Variability perpendicular to the UCM (in the **orthogonal complement**) directly affects task performance.

Geometrically, $\text{UCM}(\mathbf{q}_0)$ is the kernel of the task map's differential. Where J_h has full rank k , the implicit function theorem makes h a submersion near $\mathbf{q}_0$, so the level set $h^{-1}(\mathbf{y}_0)$ is a smooth $(n - k)$ -dimensional manifold whose tangent space at $\mathbf{q}_0$ is exactly the UCM. Motion along that manifold changes the configuration without changing the task variable.

It is worth being explicit that this is a statement about the *task map*, not about the dynamics. The UCM is a redundancy of h , not an uncontrollable subspace of the control system: a configuration can be fully controllable and still have a large uncontrolled manifold, because controllability asks what states are reachable while the UCM asks which state changes the task cannot see. Conflating the two makes the hypothesis sound like a claim about what the nervous system *cannot* do, when it is a claim about what it need not bother to correct.

In Plain Language 1.1. Mathematical Reminder: The Geometry of the UCM

Recall our differential geometry from Volume 0: a manifold is a curved surface representing valid states. The **Uncontrolled Manifold** (UCM) is quite literally a geometric submanifold embedded inside the larger configuration space $\mathcal{Q}$. Because the task geometry is non-linear (e.g., the arc of a swinging robot arm), the UCM is curved. We calculate its flat **Tangent Space** mathematically using the null space of the Jacobian ($J_h \delta\mathbf{q} = \mathbf{0}$). As long as the nervous system's noise vectors point exclusively into this tangent space, the physical position of the hand will not

budge from the target. The nervous system is actively shaping its noise ellipsoid to align with the tangent space of the target manifold.

The UCM hypothesis predicts that the nervous system:

1. **Suppresses** variability in the orthogonal complement of the UCM (task-relevant variability)
2. **Tolerates** or even **encourages** variability within the UCM (task-irrelevant variability)

This has been experimentally confirmed across many motor tasks [35, 41].

```

1 import numpy as np
2 from typing import Tuple
3
4 def ucm_analysis(
5     joint_angles: np.ndarray,
6     task_jacobian_func,
7     n_repetitions: int | None = None
8 ) -> Tuple[float, float, float]:
9     """Perform UCM variance decomposition.
10
11     Parameters
12     -----
13     joint_angles : np.ndarray, shape (n_reps, n_dof)
14         Joint angle data from multiple repetitions of the same task.
15     task_jacobian_func : callable
16         q -> J(q), shape (k, n_dof), the task Jacobian.
17     n_repetitions : int, optional
18         Number of repetitions (inferred from data if not given).
19
20     Returns
21     -----
22     Tuple[float, float, float]
23         (variance_ucm, variance_ort, ratio)
24         UCM variance, orthogonal variance, and their ratio.
25     """
26     n_reps, n_dof = joint_angles.shape
27     q_mean = np.mean(joint_angles, axis=0)
28
29     # Task Jacobian at the mean configuration
30     J = task_jacobian_func(q_mean)
31     k = J.shape[0] # task dimension
32
33     # Null space of J (UCM directions)
34     U, S, Vt = np.linalg.svd(J, full_matrices=True)
35     rank = np.sum(S > 1e-10)
36     null_space = Vt[rank:].T # Shape: (n_dof, n_dof - rank)
37
38     # Orthogonal complement (range space)
39     range_space = Vt[:rank].T # Shape: (n_dof, rank)
40
41     # Decompose variability
42     deviations = joint_angles - q_mean # (n_reps, n_dof)
43
44     # Project onto UCM
45     proj_ucm = deviations @ null_space # (n_reps, n_dof - rank)
46     var_ucm = np.mean(np.sum(proj_ucm**2, axis=1))

```

```

47
48     # Project onto orthogonal complement
49     proj_ort = deviations @ range_space # (n_reps, rank)
50     var_ort = np.mean(np.sum(proj_ort**2, axis=1))
51
52     # Normalize by dimensionality
53     var_ucm_per_dof = var_ucm / (n_dof - rank) if n_dof > rank else 0
54     var_ort_per_dof = var_ort / rank if rank > 0 else 0
55
56     ratio = var_ucm_per_dof / var_ort_per_dof if var_ort_per_dof > 0 else np.
57     inf
58
59     return var_ucm_per_dof, var_ort_per_dof, ratio
60
61 # Example: 3-joint planar arm reaching to a point
62 def planar_arm_jacobian(q: np.ndarray, L: np.ndarray = np.array([0.3, 0.25,
63     0.2])):
64     """Task Jacobian for endpoint position of a 3-link planar arm."""
65     n = len(q)
66     J = np.zeros((2, n))
67     for i in range(n):
68         for j in range(i, n):
69             angle_sum = np.sum(q[:j+1])
70             J[0, i] -= L[j] * np.sin(angle_sum)
71             J[1, i] += L[j] * np.cos(angle_sum)
72
73     return J
74
75 # Generate synthetic reaching data (50 repetitions)
76 np.random.seed(42)
77 q_nominal = np.array([0.5, 1.2, -0.3])
78 n_reps = 50
79 # Motor noise: more variability in task-irrelevant directions
80 noise = np.random.randn(n_reps, 3) * 0.05
81 q_data = q_nominal + noise
82
83 v_ucm, v_ort, ratio = ucm_analysis(q_data, planar_arm_jacobian)
84 print(f"UCM variance (per DOF): {v_ucm:.6f}")
85 print(f"ORT variance (per DOF): {v_ort:.6f}")
86 print(f"UCM/ORT ratio: {ratio:.2f}")
87 print(f"(Ratio > 1 supports the UCM hypothesis)")

```

Listing 1.1: Uncontrolled manifold analysis.

1.4 Bernstein's Three Stages of Learning

Bernstein proposed that motor skill acquisition progresses through three stages:

1. **Freezing:** initially, the learner constrains (“freezes”) excess degrees of freedom, reducing the system to manageable dimensionality. A beginner golfer keeps wrists rigid, eliminating the wrist DOF.
2. **Freeing:** as skill develops, the learner gradually releases (“frees”) the frozen DOF, allowing them to participate in the movement. The intermediate golfer adds wrist hinge.

3. **Exploiting:** the expert learner exploits the released DOF to harness reactive forces, elastic energy, and passive dynamics. The expert golfer uses the passive wrist release driven by centrifugal force.

In the language of control theory:

- Stage 1: reduce the system dimension from n to k (the task dimension)
- Stage 2: increase the controlled dimension while maintaining task performance
- Stage 3: exploit the drift dynamics $f(\mathbf{x})$ by designing trajectories that use passive mechanics (Volume II, Chapter 5). Agrachev & Sachkov [1] develop the geometric theory of reachable sets in Chapter 8; what a system can reach depends on its drift as much as on its inputs, which is why releasing a degree of freedom can enlarge the reachable set without adding an actuator.

Exercises

1. **Redundancy Quantification.** For each of the following tasks, estimate the task dimension k , body DOF n , and compute the kinematic redundancy: (a) Standing balance, (b) Picking up a cup, (c) Walking on a treadmill, (d) A tennis serve.
2. **UCM Analysis.** Using the provided code, generate synthetic data for 3-joint planar reaching with structured variability (more noise in the null space of the Jacobian). Verify that the UCM/ORT ratio exceeds 1.
3. **Bernstein's Stages.** Observe a beginner and an expert performing the same motor task (e.g., throwing a ball). Describe their movement in terms of Bernstein's three stages. Which degrees of freedom are frozen, freed, and exploited?
4. **(Programming.)** Implement a UCM analysis for a 7-DOF arm model reaching to a 3D target. Visualize the UCM and ORT directions.
5. **(Research.)** Read Scholz and Schöner (1999) and summarize the key experimental evidence for the UCM hypothesis.

Chapter 2

The Curse of Dimensionality — And How Humans Solve It

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

In ten dimensions, the unit hypercube has $2^{10} = 1024$ corners. In a hundred, it has $2^{100} \approx 10^{30}$. The nervous system controls hundreds of dimensions every second.

2.1 Classical Curse of Dimensionality

The curse of dimensionality [4] states that the computational resources needed to solve many problems grow *exponentially* with the state dimension n :

Definition 2.1. The Curse of Dimensionality

For a control problem with state dimension n :

- **Grid-based methods** (dynamic programming): cost $\sim \mathcal{O}(N^n)$ where N is the grid resolution per dimension
- **Sample-based methods** (Monte Carlo, RL): sample complexity grows polynomially in n but with large constants
- **Trajectory optimization** (DDP, collocation): cost $\sim \mathcal{O}(n^3 \cdot T)$ per iteration — the only scalable approach for deterministic systems

2.2 Biological Solutions

Humans solve the dimensionality problem through five complementary mechanisms:

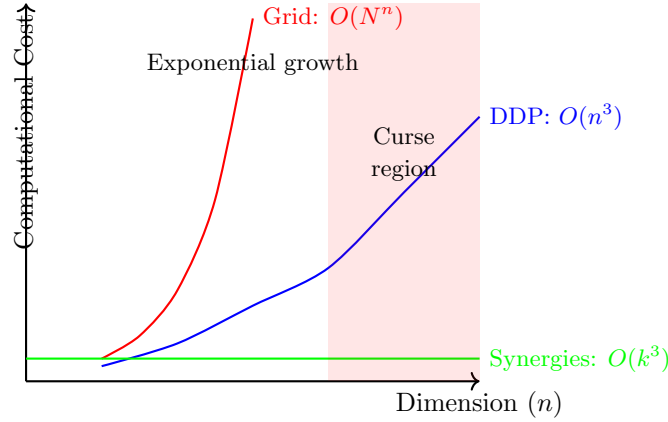


Figure 2.1: The curse of dimensionality: computational cost grows exponentially with dimension for grid-based methods, cubically for DDP, and remains constant when using synergy-based dimensionality reduction (where $k \ll n$).

2.2.1 1. Synergy-Based Compression

As introduced in Chapter 4: $\mathbf{u}(t) = W\mathbf{c}(t)$ reduces p -dimensional muscle space to k -dimensional synergy space, with $k/p \approx 4/400 = 1\%$.

2.2.2 2. Hierarchical Decomposition

Multi-level processing decomposes the global problem into independent subproblems.

2.2.3 3. Passive Dynamics Exploitation

The drift field $f(\mathbf{x})$ does useful work. The controller only corrects deviations. This reduces the effective control dimension (Volume II, passive dynamics).

2.2.4 4. Internal Models and Prediction

Forward models predict consequences, enabling open-loop execution with sparse feedback corrections (see Chapter 6).

2.2.5 5. Task-Space Control

Control in a low-dimensional task space rather than the full joint/muscle space. The null space absorbs variability (UCM framework, Chapter 1).

```

1 import numpy as np
2
3 def computational_cost_table(n_dims: list[int]) -> None:
4     """Print computational cost for different methods vs dimension."""
5     print(f"{'n_dim':>6} {'Grid (N=10)':>15} {'DDP (T=100)':>15} "
6           f"{'Synergy (k=5)':>15}")
7     print("-" * 55)
8     for n in n_dims:

```



```
9     grid_cost = 10**n if n <= 10 else float('inf')
10     ddp_cost = n**3 * 100
11     synergy_cost = 5**3 * 100 # After reduction to k=5
12     print(f"{n:>6d} {grid_cost:>15.0e} {ddp_cost:>15.0e} "
13           f"{synergy_cost:>15.0e}")
14
15 computational_cost_table([2, 5, 10, 20, 50, 100, 200])
```

Listing 2.1: Computational cost comparison across methods.

Exercises

1. **Scaling Analysis.** Using the computational cost table, determine the maximum state dimension for which grid-based dynamic programming is feasible (assuming 10^{12} FLOPS).
2. **Synergy Efficiency.** If a motor task uses 100 muscles with 5 synergies, by what factor does the synergy decomposition reduce the dimensionality of the control problem?
3. **(Programming.)** Implement DDP for a 2-DOF, 5-DOF, and 10-DOF chain. Measure computation time per iteration. Verify cubic scaling in state dimension.

Chapter 3

Neural Architecture of Motor Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The motor system is not one controller. It is an orchestra of specialized modules, each playing its part in the symphony of movement.

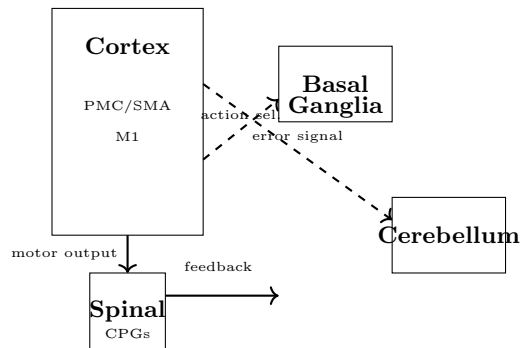


Figure 3.1: Hierarchical neural motor architecture: cortex plans/initiates, basal ganglia selects actions, cerebellum provides error-based learning, spinal cord implements CPGs and reflex circuits.

3.1 Cortical Motor Areas

The cerebral cortex contains several distinct motor areas, each with specialized roles:

3.1.1 Primary Motor Cortex (M1)

M1 (Brodmann area 4) is the final cortical output stage for voluntary movement. Layer V contains the giant pyramidal (Betz) cells whose axons form the corticospinal tract—the most direct neural pathway from cortex to spinal motor neurons.

Key properties of M1 (experimentally confirmed):

- **Somatotopic organization:** body parts are mapped to cortical regions (the motor homunculus), with overlapping representations [34]
- **Population coding:** individual neurons have broad cosine-shaped directional tuning about a preferred direction [11], and movement direction is recovered from the vector sum of the population's contributions rather than from any single cell [12]
- **Preparatory activity:** M1 neurons show activity 100–200 ms before movement onset during motor preparation, reflecting planning processes

3.1.2 Premotor Cortex (PMC) and Supplementary Motor Area (SMA)

- **PMC:** movement planning based on external cues (visually guided reaching)
- **SMA:** movement planning based on internal cues (self-initiated sequences)
- Both areas project to M1 and directly to the spinal cord

3.2 Subcortical Motor Structures

3.2.1 Cerebellum: The Forward Model

The human cerebellum contains roughly 69 billion neurons—about 80% of all the neurons in the brain, and overwhelmingly granule cells—against some 16 billion in the cerebral cortex [3]. Electrophysiological and lesion studies establish its primary role in motor control as a **forward model**:

Neural Architecture 3.1. The Cerebellum as a Forward Model

The cerebellar circuit [29, 2, 24]:

1. Receives a copy of the motor command (efference copy) via the pontine nuclei
2. Receives sensory feedback via the inferior olive and mossy fibers
3. Computes the predicted sensory consequences of the command using parallel fiber–Purkinje cell synapses (basis function expansion)
4. The climbing fiber signal carries sensory prediction error from the inferior olive
5. Error-driven learning updates Purkinje cell responses through long-term depression (LTD) at parallel fiber–Purkinje cell synapses. LTD is induced only when parallel fiber activity and the climbing fiber spike fall within a window of a few hundred milliseconds of each other; the optimum is region-specific and appears tuned to each region's own sensorimotor feedback delay [40]

In the language of control theory (Volume I):

- The cerebellum implements $\hat{\mathbf{x}}_{k+1} = f(\hat{\mathbf{x}}_k, \mathbf{u}_k)$ — the state prediction step of the Kalman filter
- The climbing fiber error signal implements the innovation update
- The granule cell layer provides the basis functions for the nonlinear mapping

Two lines of evidence support this architecture, and it is worth keeping them apart because they come from different populations.

In healthy subjects, reaching against a novel force field is initially distorted and then straightens with practice; removing the field produces aftereffects that mirror the original distortion, which is what implicates a learned internal model of the limb's dynamics rather than a stiffer or better-tuned feedback loop [37]. In patients with cerebellar degeneration, on-line correction within a movement is intact while trial-to-trial learning of those same altered dynamics is profoundly impaired—a dissociation that does not appear in Huntington's disease, where the deficit falls on the feedback side instead [38]. The first result establishes that an internal model is learned; the second locates the learning in the cerebellum.

Two timescales are easy to conflate here, and they are not the same quantity. The window in which parallel fiber and climbing fiber activity must coincide for LTD to be induced is a few hundred milliseconds—that is a coincidence-detection requirement, set by how long the error signal takes to arrive. The depression it produces, and the behavioural adaptation that follows, accumulate over minutes and tens of trials. Motor learning being slow is therefore not in tension with the plasticity rule being fast: the millisecond figure governs *whether* a synapse is eligible to change, the minutes figure governs *how much* it has changed.

3.2.2 Basal Ganglia: Action Selection

The basal ganglia implement an action selection mechanism through competitive inhibition. The direct pathway facilitates a desired action; the indirect pathway suppresses competing actions. This is analogous to a model predictive controller evaluating multiple candidate trajectories and selecting the best one.

3.3 Spinal Cord: The Local Controller

The spinal cord is not merely a relay station. It contains:

- **Motor neuron pools:** lower motor neurons that directly innervate muscles
- **Interneuron networks:** local circuits that implement reflexes, pattern generation, and coordination
- **Central pattern generators (CPGs):** oscillatory circuits for rhythmic movements (Chapter 8)

Exercises

1. **Population Vector.** Given neurons with preferred directions every 45° , compute the population vector for a movement at 30° . Plot individual neuron contributions.
2. **Cerebellar Forward Model.** Implement a simple forward model using a lookup table. Show how climbing fiber errors update the model over successive trials.
3. **(Research.)** Read Georgopoulos et al. (1986) and summarize the evidence for population coding in M1.

Chapter 4

Shallow-but-Wide: Why Brains Are Parallel, Not Deep

The brain is not a deep learning system. It is a massively parallel, notably shallow one. This is not a bug — it is the architecture that makes real-time motor control possible.

4.1 The Speed Constraint

The most fundamental constraint on neural computation is **speed**. Every biological computation has a time cost determined by the physics of synaptic transmission:

Neural Architecture 4.1. Neural Timing Constraints

Measured values from neurophysiology [17, 23]:

- **Synaptic delay:** 1–5 ms per chemical synapse (AMPA receptor rise time ~ 1 ms)
- **Axonal conduction:** unmyelinated ~ 1 m/s, myelinated ~ 100 m/s
- **Action potential:** rising phase 1 ms, falling phase 2–3 ms
- **Refractory period:** absolute ~ 1 –2 ms (sodium channel inactivation)
- **Total per-layer delay:** ~ 5 –10 ms (synaptic + integration + propagation)

These delays are not technological limitations to be overcome with faster hardware. They are *fundamental physical constraints* of electrochemical signaling. No amount of biological optimization can reduce synaptic delay below ~ 1 ms.

4.1.1 The Depth Budget

Given the timing constraints, we can compute the maximum depth of any neural computation that must complete within a given time window:

Definition 4.1. The Neural Depth Budget

For a motor task requiring a response within time T_{response} , the maximum number of serial processing layers is:

$$L_{\text{max}} = \frac{T_{\text{response}}}{\tau_{\text{layer}}}, \quad (4.1)$$

where $\tau_{\text{layer}} \approx 5\text{--}10$ ms is the per-layer processing time. For typical motor control scenarios:

Task	T_{response}	L_{max}
Spinal reflex	30 ms	3–6
Postural correction	100 ms	10–20
Voluntary movement initiation	150–250 ms	15–50
Golf swing correction (mid-downswing)	50 ms	5–10
Saccadic eye movement	200 ms	20–40

Compare this with modern deep learning: GPT-4 has ~ 120 transformer layers; ResNet-152 has 152 layers. These run on silicon at $\sim \text{ns}$ per operation. Biology cannot afford this depth at $\sim \text{ms}$ per operation.

4.2 The Motor Cortex: Six Layers, Not Sixty

The cerebral cortex has a notably consistent six-layered architecture across mammalian species [32]. This constraint applies uniformly to M1 and other cortical areas:

1. **Layer I:** molecular layer (mostly dendrites and axons, few cell bodies)
2. **Layer II/III:** external pyramidal layer (cortico-cortical connections)
3. **Layer IV:** internal granular layer (thalamic input)
4. **Layer V:** internal pyramidal layer (*output layer* — corticospinal tract)
5. **Layer VI:** multiform layer (feedback to thalamus)

The signal path from sensory input (Layer IV) through cortical processing to motor output (Layer V) passes through *at most 3–4 synapses within the cortex itself*. Adding thalamic relay and spinal motor neuron connections, the total sensorimotor loop is approximately 8–12 synapses deep.

Key Insight 4.1. The Architecture Is the Algorithm

In deep learning, we can stack layers because silicon is fast. In biology, we cannot. Instead, the brain uses:

1. **Width:** approximately 16 billion cortical neurons in humans [3], each receiving on the order of 10^4 synaptic inputs [23]. The total computational capacity is

in the connections, not the depth.

2. **Recurrence:** horizontal connections within layers create recurrent dynamics that effectively increase computational depth through temporal iteration (but at the cost of time). These recurrent loops operate at 10–100 ms timescales within a single motor control cycle.
3. **Specialization:** different brain regions process different aspects of the motor task in parallel, communicating through thick fiber bundles (corpus callosum, cerebral peduncles).
4. **Hierarchy:** a shallow hierarchy of cortex → subcortical structures (basal ganglia, cerebellum, brainstem) → spinal cord → muscles, with each level operating at different timescales and abstraction levels.

This architectural choice has far-reaching consequences. While a deep network might process sensory information in 50 serial steps, the brain achieves comparable flexibility through width and recurrence. Consider a reaching task: the cortex does not recompute the entire motor plan from scratch at each millisecond. Instead, it maintains a dynamical state that evolves according to its intrinsic connectivity. Perturbations are handled through feedback loops that operate in parallel at multiple levels.

Deep: 10 layers × 50 wide

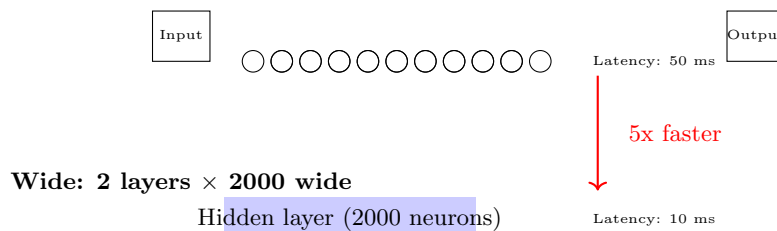


Figure 4.1: Shallow-wide architecture trades depth for width: a 2-layer network with 2000 neurons per layer completes in 10 ms, while a 10-layer network with 50 neurons per layer takes 50 ms—too slow for real-time motor control.

4.3 Computational Implications

The shallow-but-wide architecture has notable implications for motor control theory:

4.3.1 Complex Computations Through Simple Local Rules

Each neuron performs a relatively simple computation: weighted sum of inputs, passed through a nonlinear activation function. The power of the system comes from the *collective behavior* of billions of these simple units operating in parallel.


```

1 import numpy as np
2 import time
3
4 def deep_network_forward(
5     x: np.ndarray,
6     weights: list[np.ndarray],
7     biases: list[np.ndarray],
8     layer_delay_ms: float = 5.0
9 ) -> tuple[np.ndarray, float]:
10     """Forward pass through a deep network with biological timing.
11
12     Parameters
13     -----
14     x : np.ndarray
15         Input vector.
16     weights : list of np.ndarray
17         Weight matrices for each layer.
18     biases : list of np.ndarray
19         Bias vectors for each layer.
20     layer_delay_ms : float
21         Simulated per-layer delay in milliseconds.
22
23     Returns
24     -----
25     tuple
26         (output, total_latency_ms)
27     """
28     h = x
29     total_latency = 0.0
30     for W, b in zip(weights, biases):
31         h = np.maximum(0, W @ h + b) # ReLU activation
32         total_latency += layer_delay_ms
33     return h, total_latency
34
35
36 def wide_network_forward(
37     x: np.ndarray,
38     W_hidden: np.ndarray,
39     b_hidden: np.ndarray,
40     W_out: np.ndarray,
41     b_out: np.ndarray,
42     layer_delay_ms: float = 5.0
43 ) -> tuple[np.ndarray, float]:
44     """Forward pass through a wide (2-layer) network.
45
46     Parameters
47     -----
48     x : np.ndarray
49         Input vector.
50     W_hidden : np.ndarray
51         Hidden layer weights (wide).
52     b_hidden : np.ndarray
53         Hidden layer biases.
54     W_out : np.ndarray
55         Output layer weights.
56     b_out : np.ndarray
57         Output biases.
58     layer_delay_ms : float
59         Per-layer delay.
60

```

```

61     Returns
62     -----
63     tuple
64     (output, total_latency_ms)
65     """
66     h = np.maximum(0, W_hidden @ x + b_hidden)
67     y = W_out @ h + b_out
68     total_latency = 2 * layer_delay_ms # Only 2 layers
69     return y, total_latency
70
71
72 # Compare architectures for a motor control task
73 # Input: 20-dim state, Output: 10-dim muscle activations
74 np.random.seed(42)
75 n_input, n_output = 20, 10
76 n_params_budget = 50000 # Same total parameter budget
77
78 # Deep network: 10 layers of width 50
79 deep_widths = [n_input] + [50] * 9 + [n_output]
80 deep_W = [np.random.randn(deep_widths[i+1], deep_widths[i]) * 0.1
81           for i in range(len(deep_widths)-1)]
82 deep_b = [np.zeros(deep_widths[i+1]) for i in range(len(deep_widths)-1)]
83
84 # Wide network: 2 layers, width 2000
85 wide_hidden = 2000
86 W_h = np.random.randn(wide_hidden, n_input) * 0.1
87 b_h = np.zeros(wide_hidden)
88 W_o = np.random.randn(n_output, wide_hidden) * 0.1
89 b_o = np.zeros(n_output)
90
91 x = np.random.randn(n_input)
92
93 y_deep, latency_deep = deep_network_forward(x, deep_W, deep_b)
94 y_wide, latency_wide = wide_network_forward(x, W_h, b_h, W_o, b_o)
95
96 print(f"Deep network (10 layers x 50 wide):")
97 print(f"  Latency: {latency_deep:.0f} ms")
98 print(f"  Parameters: {sum(W.size + b.size for W, b in zip(deep_W, deep_b))
99       : ,}")
100
101 print(f"\nWide network (2 layers x 2000 wide):")
102 print(f"  Latency: {latency_wide:.0f} ms")
103 print(f"  Parameters: {W_h.size + b_h.size + W_o.size + b_o.size: ,}")
104
105 print(f"\nLatency ratio: {latency_deep/latency_wide:.1f}x faster with wide
106       network")

```

Listing 4.1: Comparison: deep vs. wide network for motor control.

4.3.2 Hierarchical Decomposition

The brain does not solve the full-dimensional control problem in one computation. Instead, it decomposes the problem hierarchically:

1. **High level** (prefrontal cortex, ~200 ms): task goal, strategy
2. **Mid level** (premotor/SMA, ~100 ms): movement plan, sequencing
3. **Low level** (M1 + spinal cord, ~30 ms): muscle activation patterns

Each level operates on a compressed representation of the level below:

Proposition 4.1. *In a hierarchical motor control architecture with L levels, each reducing dimensionality by a factor r , the effective state dimension at level l is:*

$$n_l = n_0 / r^l, \quad (4.2)$$

where n_0 is the full state dimension. The computational cost per level scales as $\mathcal{O}(n_l^2)$ for a single-layer network, giving total cost $\sum_{l=0}^L n_l^2 = n_0^2 \sum_{l=0}^L r^{-2l}$, which converges for $r > 1$. The curse of dimensionality is broken by hierarchy.

4.3.3 Synergies as Dimensionality Reduction

Muscle synergies provide direct evidence for dimensionality reduction. d’Avella and Bizzi showed that the muscle patterns of natural frog hindlimb behaviours decompose into a small set of synergies, some shared across behaviours and some specific to one [10]. How far that conclusion depends on the decomposition used is a separate question, and Tresch and colleagues addressed it directly by running several matrix factorization algorithms against the same simulated and experimental data: the algorithms broadly agree on the synergies they recover, which is what makes the finding a property of the data rather than of the method [42]. The muscle activation pattern of p muscles is approximated as a linear combination of $k \ll p$ synergies:

$$\mathbf{u}(t) = \sum_{i=1}^k c_i(t) \mathbf{w}_i = W \mathbf{c}(t), \quad (4.3)$$

where $W \in \mathbb{R}^{p \times k}$ is the synergy matrix (fixed spatial patterns) and $\mathbf{c}(t) \in \mathbb{R}^k$ are the time-varying synergy activations.

Experimental studies across diverse tasks (reaching, walking, postural responses) consistently identify $k = 4$ – 6 synergies accounting for $> 90\%$ variance (Tresch et al. 2006, Ting and Macpherson 2005).

```

1 import numpy as np
2
3 def extract_synergies(
4     emg_data: np.ndarray,
5     n_synergies: int,
6     max_iter: int = 500,
7     tol: float = 1e-6
8 ) -> tuple[np.ndarray, np.ndarray, float]:
9     """Extract muscle synergies using NMF.
10
11     Parameters
12     -----
13     emg_data : np.ndarray, shape (n_timepoints, n_muscles)
14         Processed EMG data (non-negative).
15     n_synergies : int
16         Number of synergies to extract.
17     max_iter : int
18         Maximum NMF iterations.
19     tol : float
20         Convergence tolerance.
21
22     Returns

```

```

23     -----
24     tuple
25         (synergy_weights, synergy_activations, vaf)
26         W: (n_muscles, n_synergies)
27         C: (n_timepoints, n_synergies)
28         vaf: variance accounted for (0-1)
29     """
30     n_t, n_m = emg_data.shape
31
32     # Initialize with random non-negative values
33     W = np.abs(np.random.randn(n_m, n_synergies)) * 0.01
34     C = np.abs(np.random.randn(n_t, n_synergies)) * 0.01
35
36     for iteration in range(max_iter):
37         # Multiplicative update rules (Lee & Seung 2001)
38         # Update C
39         numerator = emg_data @ W
40         denominator = C @ W.T @ W + 1e-10
41         C *= numerator / denominator
42
43         # Update W
44         numerator = emg_data.T @ C
45         denominator = W @ C.T @ C + 1e-10
46         W *= numerator / denominator
47
48         # Check convergence
49         reconstruction = C @ W.T
50         residual = np.sum((emg_data - reconstruction)**2)
51         total = np.sum(emg_data**2)
52         vaf = 1.0 - residual / total
53
54         if iteration > 0 and abs(vaf - prev_vaf) < tol:
55             break
56         prev_vaf = vaf
57
58     return W, C, vaf
59
60
61 # Example: simulate EMG from 8 muscles during walking
62 np.random.seed(42)
63 n_timepoints = 200
64 n_muscles = 8
65
66 # Ground truth: 3 synergies generating 8-muscle activation
67 true_W = np.array([
68     [0.8, 0.6, 0.1, 0.0, 0.2, 0.0, 0.1, 0.0], # Syn 1: extensors
69     [0.0, 0.1, 0.7, 0.9, 0.0, 0.3, 0.0, 0.1], # Syn 2: flexors
70     [0.2, 0.0, 0.1, 0.0, 0.8, 0.7, 0.6, 0.9], # Syn 3: stabilizers
71 ]).T # (8 muscles, 3 synergies)
72
73 t = np.linspace(0, 2*np.pi, n_timepoints)
74 true_C = np.column_stack([
75     np.maximum(0, np.sin(t)),
76     np.maximum(0, np.sin(t + 2*np.pi/3)),
77     np.maximum(0, np.sin(t + 4*np.pi/3)),
78 ])
79
80 emg_simulated = true_C @ true_W.T + np.abs(np.random.randn(n_timepoints,
81     n_muscles)) * 0.05

```

```

82 # Extract synergies
83 W_est, C_est, vaf = extract_synergies(emg_simulated, n_synergies=3)
84 print(f"Synergy extraction: {vaf:.1%} variance accounted for with 3 synergies")
85
86 # Test with different numbers
87 for k in range(1, 7):
88     _, _, vaf_k = extract_synergies(emg_simulated, n_synergies=k)
89     print(f"  k={k}: VAF = {vaf_k:.1%}")

```

Listing 4.2: Muscle synergy extraction using Non-negative Matrix Factorization.

4.4 The Shallow-Wide Hypothesis for Motor Control

We now formalize the central thesis of this volume:

Key Insight 4.2. The Shallow-Wide Hypothesis

Biological motor control systems achieve high-dimensional control through architectures that are:

1. **Shallow:** at most 10–20 serial processing layers (limited by synaptic delay)
2. **Wide:** millions to billions of units per layer (enabled by neural density and parallel wiring)
3. **Modular:** functionally specialized subnetworks process different aspects of the task in parallel
4. **Hierarchical:** a small number of levels, each compressing dimensionality
5. **Recurrent:** intra-layer connections provide temporal depth without adding spatial depth

This architecture can approximate any smooth mapping (universal approximation theorem) with a single hidden layer of sufficient width. The practical advantage over deep networks is **latency**: a 2-layer network with 10,000 neurons per layer completes in 10 ms. A 100-layer network with 100 neurons per layer completes in 500 ms—too slow for motor control.

For real-time motor control, width is cheap and depth is expensive.

Exercises

1. **Depth Budget.** For a goalkeeper diving to save a penalty kick (reaction time ~ 200 ms), compute the maximum neural depth. How deep can the “controller” be?
2. **Width vs. Depth.** Using the provided code, compare the outputs of a 10-layer $\times$ 50-wide network with a 2-layer $\times$ 2000-wide network on random inputs. Compare latencies.

3. **Synergy Analysis.** Using the NMF code, determine how many synergies are needed to explain $> 95\%$ of variance in the simulated EMG data. Does this match the literature value of 4–6?
4. **(Programming.)** Implement a motor control task (reaching) using both a deep and a wide network architecture. Compare tracking accuracy at different latency budgets.
5. **(Research.)** Find experimental evidence for the number of cortical layers involved in processing a motor command from M1 to spinal motor neurons. How many synapses does the signal cross?

Chapter 5

Ideomotor Theory and the Predictive Brain

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Every representation of a movement awakens in some degree the actual movement which is its object.

— William James, 1890

5.1 Historical Ideomotor Theory

The ideomotor principle, articulated by William James in 1890 [22], states that the mere *idea* of a movement tends to produce that movement. Modern reformulations by Greenwald (1970) and Hommel et al. (2001) have transformed this philosophical observation into a testable scientific framework.

Definition 5.1. The Theory of Event Coding (TEC)

The Theory of Event Coding [19] proposes that:

1. Perception and action share a **common representational format** (event codes or feature codes)
2. Actions are planned in terms of their **desired sensory effects**, not in terms of muscle commands
3. The mapping from desired effects to motor commands is learned through experience (bidirectional action-effect associations)

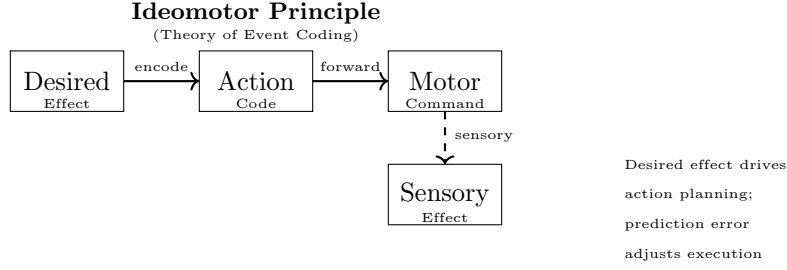


Figure 5.1: The ideomotor principle: actions are planned in terms of their desired sensory effects rather than muscle commands. The forward model predicts the sensory consequence, and prediction error corrects the action.

5.2 The Predictive Processing Framework

Modern neuroscience interprets the brain as a **prediction machine** (Clark, 2013; Hohwy, 2013):

Key Insight 5.1. The Brain as a Prediction Machine

The brain continuously generates predictions about incoming sensory data. Neural processing is dominated by **prediction error minimization**:

$$\varepsilon_k = \mathbf{y}_k - \hat{\mathbf{y}}_k = \mathbf{y}_k - h(\hat{\mathbf{x}}_k), \quad (5.1)$$

where $\hat{\mathbf{y}}_k$ is the predicted observation and $\mathbf{y}_k$ is the actual observation. This error drives updates to the internal model.

In the language of Volume I, Chapter 6: this is exactly the **Kalman filter innovation**. The brain implements Bayesian state estimation.

5.2.1 Active Inference

Karl Friston’s active inference framework (Friston, 2010) extends predictive processing to action: instead of only updating beliefs to match observations, the agent acts to make observations match predictions. This formalizes the ideomotor principle in terms of Free Energy minimization:

$$\mathbf{u}^* = \arg \min_{\mathbf{u}} \mathbb{E} \left[\sum_{k=0}^T \|\mathbf{y}_k - \hat{\mathbf{y}}_k\|_{R^{-1}}^2 + \|\mathbf{u}_k\|_Q^2 \right], \quad (5.2)$$

which is formally equivalent to model predictive control (Volume I, Chapter 5).

5.3 Connection to Control Theory

Neuroscience Term	Control Theory Term
Forward model	State transition matrix / predictor
Prediction error	Innovation (Kalman filter)
Efference copy	Feedforward control signal
Active inference	Model predictive control
Prior beliefs	State constraints / initial conditions
Precision weighting	Kalman gain / observation weights
Free energy	Variational cost functional

Exercises

1. **Conceptual.** Explain how the ideomotor principle relates to trajectory-centric control (Volume II). When a golfer imagines impact, what “prediction” are they generating?
2. **Kalman Connection.** Write out the Kalman filter equations and identify which term corresponds to the prediction error in the predictive processing framework.
3. **(Programming.)** Implement an active inference agent that controls a pendulum by minimizing prediction error rather than tracking a reference.

Chapter 6

Internal Models: Forward and Inverse

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

To control is to predict. To predict is to have a model. To have a model is to have learned.

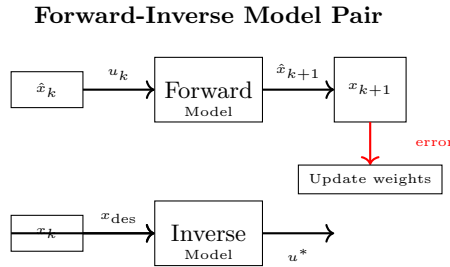


Figure 6.1: Paired forward and inverse models: forward model predicts state evolution from command; inverse model computes command to reach desired state. Learning updates both based on sensory prediction error.

6.1 Forward Models

A forward model predicts the sensory consequences of a motor command:

$$\hat{\mathbf{x}}_{k+1} = f(\hat{\mathbf{x}}_k, \mathbf{u}_k), \quad (6.1)$$

which is exactly the state propagation used in Volume I.

6.1.1 The MOSAIC Architecture

The MOSAIC model (Modular Selection and Identification for Control) [43] proposes that the brain maintains multiple paired forward-inverse models, each tuned to different dynamics or contexts. The claim that a model is acquired *per tool* rather than adjusted in place has direct imaging support: cerebellar activity acquired while learning a novel visuomotor tool persists as a tool-specific representation, separable from the activity driven by the movement itself [21].

```

1 import numpy as np
2 from typing import List, Tuple
3
4 class ForwardInversePair:
5     """A paired forward-inverse model for one context."""
6     def __init__(self, A: np.ndarray, B: np.ndarray):
7         self.A = A # State transition
8         self.B = B # Input matrix
9
10    def forward_predict(self, x: np.ndarray,
11                       u: np.ndarray) -> np.ndarray:
12        """Predict next state."""
13        return self.A @ x + self.B @ u
14
15    def inverse(self, x: np.ndarray,
16               x_desired: np.ndarray) -> np.ndarray:
17        """Compute control to reach desired state."""
18        return np.linalg.lstsq(self.B,
19                               x_desired - self.A @ x,
20                               rcond=None)[0]
21
22
23 class MOSAIC:
24     """Multiple paired forward-inverse model architecture."""
25     def __init__(self, models: List[ForwardInversePair]):
26         self.models = models
27         self.n_models = len(models)
28         self.responsibilities = np.ones(self.n_models) / self.n_models
29
30    def update_responsibilities(self, x: np.ndarray,
31                              u: np.ndarray,
32                              x_next: np.ndarray) -> None:
33        """Update model responsibilities based on prediction errors."""
34        errors = np.zeros(self.n_models)
35        for i, model in enumerate(self.models):
36            prediction = model.forward_predict(x, u)
37            errors[i] = np.sum((x_next - prediction)**2)
38
39        # Softmax weighting (lower error = higher responsibility)
40        log_resp = -0.5 * errors
41        log_resp -= np.max(log_resp) # numerical stability
42        self.responsibilities = np.exp(log_resp)
43        self.responsibilities /= np.sum(self.responsibilities)
44
45    def predict(self, x: np.ndarray,
46               u: np.ndarray) -> np.ndarray:
47        """Weighted prediction across all models."""
48        prediction = np.zeros_like(x)
49        for i, model in enumerate(self.models):
50            prediction += self.responsibilities[i] * \
51                model.forward_predict(x, u)

```

```

52     return prediction
53
54     def control(self, x: np.ndarray,
55                 x_desired: np.ndarray) -> np.ndarray:
56         """Weighted inverse model output."""
57         u = np.zeros(self.models[0].B.shape[1])
58         for i, model in enumerate(self.models):
59             u += self.responsibilities[i] * \
60                 model.inverse(x, x_desired)
61     return u

```

Listing 6.1: MOSAIC-style multiple model architecture.

6.2 Inverse Models

An inverse model computes the motor command required to achieve a desired state:

$$\mathbf{u}_k = f^{-1}(\mathbf{x}_k, \mathbf{x}_{k+1}^{\text{des}}), \quad (6.2)$$

which corresponds to the feedforward control of Volume II. How such a model could be *acquired* rather than assumed is the contribution of feedback-error learning: the output of a crude feedback controller is used as the training signal for the inverse model, so the feedback loop teaches the feedforward path and then falls silent [24]. Note that this is the inverse-model result; the forward-model evidence is separate, and conflating the two is easy because both trace to the same group.

6.3 Smith Predictor for Neural Delays

The sensorimotor system faces finite neural and sensing delays. The Smith predictor architecture uses a forward model to compensate:

$$\hat{\mathbf{x}}_{\text{current}} = \hat{\mathbf{x}}(t - \Delta t) + \int_{t-\Delta t}^t f(\hat{\mathbf{x}}, \mathbf{u}) d\tau, \quad (6.3)$$

where Δt is the sensory delay. The controller operates on the *predicted current state* rather than the delayed sensory measurement.

Exercises

1. **MOSAIC.** Using the provided code, simulate a system that switches between two dynamics (e.g., carrying an empty cup vs. a full cup). Show how the responsibilities shift.
2. **Smith Predictor.** Implement a Smith predictor for a simple pendulum with a user-chosen sensory delay parameter. Compare performance with and without prediction.
3. **(Research.)** Summarize evidence for multiple internal models in human motor control, citing at least three experimental studies.

Chapter 7

Passive Distributed Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The most efficient movement is one where the body's own mechanics do most of the work, and the nervous system merely guides.

— Adapted from Frans Bosch

7.1 The Frans Bosch Framework

Frans Bosch's work on strength training and coordination [6] provides a practitioner-oriented framework for understanding how elite athletes organize movement. His key insights, translated into the language of this series:

Key Insight 7.1. Self-Organization in Complex Movement

Bosch argues that complex movements are not centrally programmed but **self-organize** through the interaction of:

1. **Task constraints:** the goal shapes the solution space
2. **Organismic constraints:** the body's anatomy and current state
3. **Environmental constraints:** gravity, ground, implements

The role of the nervous system is not to specify every muscle activation but to set the *initial conditions* and *constraints* that allow self-organization to produce the desired movement.

In the language of Volume I: the CNS designs the *drift field* $f(\mathbf{x})$ of the system (through postural configuration and muscle tone) and lets the natural dynamics do the work.

Passive Dynamic Walking

(McGeer, 1990)

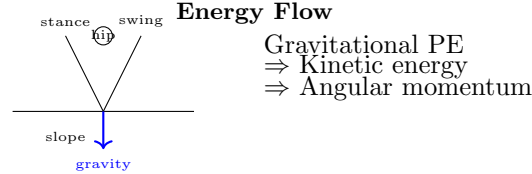


Figure 7.1: Passive dynamic walking: a compass gait on a gentle slope exhibits stable walking cycles with zero control input. Gravity and inertia provide all energy; the walker merely guides the descent.

7.2 Passive Dynamic Walking

The most striking example of passive dynamics is passive dynamic walking (McGeer, 1990): a simple mechanical biped with knees, feet, and hip (no motors) walks down a gentle slope in stable limit cycles with zero active control. This demonstrates that complex coordinated movement can emerge from mechanical design and gravity alone:

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 def passive_walker_dynamics(
5     t: float,
6     state: np.ndarray,
7     M: float = 10.0,
8     m: float = 5.0,
9     l: float = 1.0,
10    g: float = 9.81,
11    slope: float = 0.05
12 ) -> np.ndarray:
13     """Dynamics of a 2-DOF passive walker (compass gait).
14
15     Parameters
16     -----
17     t : float
18         Time.
19     state : np.ndarray
20         [theta_stance, theta_swing, d_theta_stance, d_theta_swing]
21     M : float
22         Hip mass (kg).
23     m : float
24         Leg mass at foot (kg).
25     g : float
26         Gravitational acceleration (m/s^2).
27     slope : float
28         Ground slope (radians).
29
30     Returns
31     -----
32     np.ndarray
33         State derivatives.
34     """
35     th_st, th_sw, dth_st, dth_sw = state

```

```

36
37     # Mass matrix
38     M11 = (M + m) * l**2
39     M12 = -m * l**2 * np.cos(th_st - th_sw)
40     M21 = M12
41     M22 = m * l**2
42
43     # Gravity and Coriolis
44     C1 = -m * l**2 * dth_sw**2 * np.sin(th_st - th_sw) \
45           - (M + m) * g * l * np.sin(th_st - slope)
46     C2 = m * l**2 * dth_st**2 * np.sin(th_st - th_sw) \
47           + m * g * l * np.sin(th_sw - slope)
48
49     # Solve M * ddq = C (NO control input!)
50     det = M11 * M22 - M12 * M21
51     ddth_st = (M22 * C1 - M12 * C2) / det
52     ddth_sw = (M11 * C2 - M21 * C1) / det
53
54     return np.array([dth_st, dth_sw, ddth_st, ddth_sw])
55
56
57 # Simulate one step of passive walking
58 state0 = np.array([0.2, -0.4, -0.5, 1.0])
59 sol = solve_ivp(passive_walker_dynamics, [0, 1.5], state0,
60               max_step=0.001, events=None)
61 print(f"Passive walker: simulated {sol.t[-1]:.2f}s "
62       f"with {len(sol.t)} timesteps")
63 print(f"Final stance angle: {sol.y[0, -1]:.3f} rad")
64 print("(No actuators, no controller --- just gravity and mechanics)")

```

Listing 7.1: Passive dynamic walker simulation.

7.3 Passive Dynamics in the Golf Swing

The golf swing exemplifies passive dynamics exploitation:

1. **Backswing:** energy stored in elastic tissues (muscles, tendons, fascia)
2. **Transition:** elastic recoil initiates the downswing passively
3. **Wrist release:** centrifugal force passively uncocks the wrists (the double pendulum effect from Volume II, Chapter 5)
4. **Follow-through:** passive deceleration through tissue compliance

In the ZTCF family framework of Volume I, Chapter 7: the drift contribution $f(\mathbf{x})$ accounts for $\sim 60\%$ of the clubhead speed. The control contribution $G(\mathbf{x})\mathbf{u}$ provides the remaining $\sim 40\%$.

7.4 Bernstein's Stage 3: Exploiting DOF

The connection to Bernstein's learning framework (Chapter 1) is direct:

- The expert athlete has reached Stage 3 — exploiting degrees of freedom

- They have learned to configure the body so that passive dynamics *automatically* produce the desired motion
- The control problem is simplified: instead of commanding every muscle, set up the initial conditions and let physics do the work

Exercises

1. **Passive Walking.** Using the provided code, find the set of initial conditions that produce a stable limit cycle (periodic gait).
2. **Energy Analysis.** For the passive walker, plot kinetic and potential energy over one stride. Where does the energy come from?
3. **Wrist Release.** Model the wrist release as a passive double pendulum. For what initial angular velocity of the arm does the club reach maximum speed at the bottom?
4. **(Programming.)** Add a small actuator to the passive walker and design a minimal controller that stabilizes the gait on level ground.

Chapter 8

The Interplay of Biology and Dynamics of Nonlinear Systems

8.1 Introduction: Mechanics With a Biological State

A golfer can arrive at nearly the same joint configuration and velocity with different muscle activation, tendon strain and sensory estimates. Those states need not produce the same subsequent motion. Geometry and rigid-body mechanics remain essential, but positions and velocities alone may no longer close the model. The additional state determines how a neural command becomes force, how a disturbance propagates and which responses remain feasible before impact.

This is the useful connection between biology and nonlinear dynamics. Tissue properties, neural feedback, task goals and contact conditions interact through specific equations. Their interaction can support coordination; it can also produce delay, instability, fatigue and estimation error. A biological label does not decide which outcome occurs.

Key Insight 8.1. Three Questions That Must Be Kept Together

1. **Mechanics:** what forces and moments follow from the current geometry, motion, tissue state and contact conditions?
2. **Control:** which excitation or actuator histories can change that state within the remaining time and input limits?
3. **Evidence:** which measurements distinguish the proposed mechanism from another model that produces the same observed movement?

For a swing, the endpoint objective includes the clubhead's delivered position, orientation and velocity, followed by collision dynamics. For a humanoid holding a tool, it also includes grasp feasibility and balance. Neither objective is specified by a single joint torque or a single measure of movement variability.

8.1.1 Bernstein, Constraints and the Scope of the Claim

Having more muscle forces than independent joint moments makes force allocation underdetermined unless additional information or assumptions are supplied. It does not prove that the nervous system cannot coordinate the muscles, nor that it must use a fixed small number of commands. A model's degrees of freedom, actuators, measured EMG channels and task variables are different counts.

Newell grouped constraints into organism, task and environment [33]. Bosch discusses

stable and adaptable movement components in sport [7, 6]. These frameworks motivate hypotheses; formal stability results require a separate model and proof. Bosch’s “fluctuations” can denote flexible task-dependent components; they are not, by definition, expensive errors that must always be suppressed.

An intersection of feasible sets constrains possible states. It does not by itself specify a vector field, an attractor or a controller. Two systems with identical feasible configurations can have different inertia, dissipation and feedback, and therefore different trajectories. Constraints-led practice and predictive or feedback control are compatible descriptions at different levels. Whether a golfer uses a particular internal representation requires evidence beyond a successful dynamical model of the movement.

8.2 Muscle, Tendon and Activation Dynamics

8.2.1 A Consistent Hill-Type Model

Hill’s shortening relation differs from later Hill-type muscle–tendon models [16, 31]. Declare which elements are included, where their lengths are measured and how force is transmitted. In an illustrative massless equilibrium model, a contractile element and passive parallel element share the fiber length l_f . Their force is transmitted through a series tendon, with pennation angle α :

$$F_f = aF_0f_L(l_f/l_0)f_V(\dot{l}_f/v_{\max}) + F_{PE}(l_f), \quad (8.1)$$

$$F_t = F_f \cos \alpha, \quad l_{MT}(q) = l_t + l_f \cos \alpha. \quad (8.2)$$

Here $a \in [0, 1]$ is activation, F_0 the reference active isometric fiber force, l_0 the optimal fiber length, and $v_{\max} > 0$ a shortening-speed scale. The parallel force uses fiber length, not the total muscle–tendon length. Pennation requires its own geometric law if it changes. Fiber and tendon forces are not added as though series elements were parallel force sources.

A simple tension-only tendon law is

$$F_t = k_t[l_t - l_{\text{slack}}]_+, \quad [z]_+ = \max(z, 0), \quad (8.3)$$

with k_t in N/m. The extension is relative to tendon slack length, not muscle length. This piecewise linear law is a teaching approximation; physiological tendon models can include a nonlinear toe region and hysteresis. Series equilibrium and path length together determine fiber velocity or require an implicit solve. Slack tendon and near-zero activation need careful numerical treatment rather than division by a vanishing active-force term.

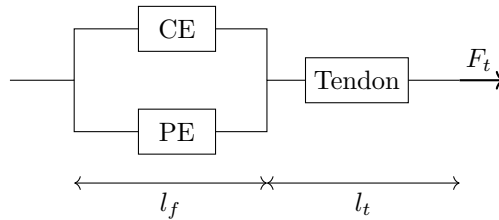


Figure 8.1: Parallel Fiber Elements and a Series Tendon. The Schematic Uses Zero Pennation; a Pennate Model Must Project Fiber Force and Length Consistently.

8.2.2 Length and Velocity Conventions That Can Be Checked

Let $\nu = \dot{l}_f/v_{\max}$, so shortening means $\nu < 0$. One explicit illustrative pair of normalized curves is

$$f_L(\ell) = \exp \left[- \left(\frac{\ell - 1}{w} \right)^2 \right], \quad w > 0, \quad (8.4)$$

$$f_V(\nu) = \begin{cases} \frac{1 + \nu}{1 - \nu/c}, & -1 \leq \nu \leq 0, \\ 1 + (N - 1) \frac{\nu}{d + \nu}, & \nu > 0, \end{cases} \quad c, d > 0, \quad N > 1. \quad (8.5)$$

These equations define the example, not a universal parameter fit or Hill's original eccentric law. They give $f_V(-1) = 0$, $f_V(0) = 1$, and an eccentric limit N . The shortening branch increases with ν ; faster shortening therefore reduces force. Its derivative at zero is $1 + 1/c$. Choosing $d = (N - 1)/(1 + 1/c)$ also matches the eccentric branch's derivative there. Outside the stated domain, choose and validate an extension rather than letting negative active tension silently enter a simulation.

For $c = 0.25$, $N = 1.5$ and $d = 0.1$, $f_V(-0.5) = 1/6$ and $f_V(0.1) = 1.25$. At $a = 0.5$, $F_0 = 1000$ N and optimal length, active fiber force is respectively 83.33 N and 625 N. These are chosen model numbers, not measured human values. Passive force and pennation still affect tendon force. Total force at a long fiber length can exceed active isometric force because the passive element contributes; the active force-length peak alone does not bound the total tension.

8.2.3 Zero Excitation Retains the Activation History

Excitation u and activation a are different variables. For a constant activation time scale $\tau_a > 0$, a simple example is

$$\dot{a} = (u - a)/\tau_a, \quad 0 \leq u \leq 1. \quad (8.6)$$

If u is set to zero at t_0 , then $a(t) = a(t_0) \exp[-(t - t_0)/\tau_a]$. Active force generally persists while activation decays. Real activation/deactivation dynamics can require different time scales and nonlinearities. With activation, fiber length and other internal states z , a closed model uses $x = (q, v, a, z)$, not only $x = (q, v)$.

For muscle path lengths $l_{\text{MT}}(q)$, define $J_l = \partial l_{\text{MT}}/\partial q$. Positive tendon tension pulls toward shorter path length, giving generalized muscle load $-J_l^T F_t$:

$$M(q)\dot{v} + h(q, v) = -J_l(q)^T F_t(x) + Q_{\text{ext}} + A(q)^T \lambda, \quad (8.7)$$

$$\dot{q} = v, \quad \dot{z} = \psi(x), \quad Av = 0. \quad (8.8)$$

Here generalized coordinates are chosen so $\dot{q} = v$; other representations need their kinematic map. h includes the declared gravity and velocity terms, and λ enforces the current independent ideal constraints. Ground friction, grasp limits and changing contact modes need additional conditions.

With constant τ_a and no other direct excitation dependence, the augmented system is control-affine: $\dot{x} = f(x) + G(x)u$. Excitation enters the activation rows; its influence on acceleration develops through the state. The drift f at $u = 0$ includes existing activation, tendon strain and constraint response. Holding a nonzero baseline command instead defines

another vector field. It is legitimate to study that field, but it must not be called fully passive or free of ongoing energetic cost.

8.2.4 Preflex Response and Tendon Energy

An intrinsic mechanical response can precede a newly evoked neural response. At a fixed activation and around a specified state, a local approximation is

$$\delta F_f \simeq K_f \delta l_f + D_f \delta \dot{l}_f. \quad (8.9)$$

The coefficients depend on operating point and history, not just activation. This approximation is often associated with the preflex concept [27]. It introduces no new neural-loop delay; that does not imply instantaneous global force transmission, an inactive nervous system, or guaranteed stability. The sign and size of incremental stiffness, tendon compliance, limb geometry and contact determine the resulting motion.

For example, a damped inverted pendulum near upright obeys

$$I\ddot{\theta} + D\dot{\theta} + (K - mgh)\theta = 0. \quad (8.10)$$

With $I > 0$ and $D > 0$, this linear model is asymptotically stable only when $K > mgh$. Positive tissue stiffness is insufficient if it is smaller than the destabilizing gravitational term. Direct human standing measurements by Loram and Lakie found intrinsic ankle stiffness insufficient for stability in their protocol [28]. This is a useful counterexample to a general claim that baseline muscle tone automatically stabilizes posture. Spinal circuits are part of the CNS; anesthesia does not imply elimination of every reflex. Neither claim is needed to explain a mechanical preflex.

For the elastic law above,

$$E_t = \frac{1}{2}k_t[l_t - l_{\text{slack}}]_+^2, \quad \dot{E}_t = F_t\dot{l}_t \quad (8.11)$$

away from the switching point. A chosen $k_t = 20,000$ N/m and 0.02 m extension store 4 J at 400 N. Releasing 4 J in 0.02 s supplies 200 W of average recoil power in this ideal example. The energy had to be loaded earlier; recoil changes the timing of power without creating energy. Fiber work, skeletal work, tendon storage and losses must close one balance. There is no universal 50% tendon contribution to jumping or striking, and an elastic element is not generally equivalent to rescaling time in a rigid-body solution.

For a golf swing, a catapult hypothesis therefore needs a measured or bounded strain history and release timing. A compliant robot actuator needs the same accounting. More compliance can improve a specified objective or impair force bandwidth, precision and contact control; the answer depends on the task.

8.3 Attractors and Coordination Transitions

8.3.1 What Stability Does and Does Not Establish

For a declared autonomous closed-loop system, we use an asymptotically stable attractor to mean a compact invariant set that is Lyapunov stable and attracts some neighborhood. Lyapunov stability means sufficiently small disturbances remain small; attraction additionally concerns convergence to the set. Definitions of more general attractors differ, so minimality and a single negative exponent should not be substituted for these conditions.

Attraction to a periodic orbit is distinct from convergence to one time-stamped trajectory on that orbit. A stable autonomous limit cycle has a neutral phase direction, with a zero Lyapunov exponent and a unit Floquet multiplier; its transverse multipliers can lie inside the unit circle. Nearby motions can retain a phase offset while approaching the same orbit. A chaotic attractor can attract trajectories to its set while separating nearby trajectories within that set. Finite-time growth, asymptotic exponents and basin size answer different questions.

These distinctions matter in golf. A repeatable swing is a finite-duration movement ending in contact, not automatically a limit cycle or equilibrium. An appropriate analysis can use a trajectory tube, terminal error, input limits and impact-event sensitivity. A phase error that is harmless for convergence to a periodic gait can be consequential when club and ball must meet at one time.

An attractor can be generated by passive mechanics, constant powered actuation, neural feedback or a robot controller. Its existence does not identify which mechanism supplies stability, how much power is consumed or whether a new task can be accommodated. A passive downhill walker draws energy from gravity; an actively stabilized posture can consume energy without moving. A finite feedforward maneuver need not be an attractor to meet an error tolerance.

Example 8.1. A Stable Orbit With a Neutral Phase

On an annulus around $r = 1$, consider $\dot{r} = 1 - r$, $\dot{\theta} = \omega > 0$. Radial deviations decay as e^{-t} , but two solutions' angular difference remains constant. The orbit $r = 1$ attracts nearby states although their time-matched Euclidean separation need not tend to zero. Transverse recovery and phase recovery must therefore be measured separately.

8.3.2 A Cost Crossing Is Not a Dynamical Bifurcation

A dynamical bifurcation changes the qualitative behavior of a vector field or return map as a parameter varies. Two metabolic cost curves crossing does not show that either gait loses dynamical stability. Even in the simple chosen costs $C_1(v) = 1 + v^2$ and $C_2(v) = 2 + v^2/2$, the crossing at $v = \sqrt{2}$ says only which algebraic cost is smaller. It supplies no equations for a gait. An observed walk–run transition can be investigated using energetic, mechanical, perceptual and stability hypotheses; matching one transition speed does not distinguish them. These teaching curves are dimensionless examples, not fitted human metabolic laws.

Likewise, a coordination variable and an experimentally adjusted “control parameter” are not necessarily a plant state and actuator input in the control-theory sense. The relationship must be derived or fitted. Frequency may alter several physiological and mechanical processes at once.

8.3.3 The Haken–Kelso–Bunz Model With Its Correct Transition

In the classic bimanual experiment, increasing cycling frequency can cause the initial anti-phase mode to switch to in-phase coordination, using the authors' homologous-muscle phase convention [14]. Symmetric movements toward and away from the body's midline can be in-phase even though the hands' world-coordinate velocities point in opposite directions. Define the measured phase before using “same direction” as a substitute.

The standard reduced deterministic phase equation is first order:

$$\dot{\phi} = -a_H \sin \phi - 2b_H \sin(2\phi) = -V'(\phi), \quad (8.12)$$

$$V(\phi) = -a_H \cos \phi - b_H \cos(2\phi), \quad a_H, b_H > 0. \quad (8.13)$$

The potential is a reduced dynamical construction, not metabolic or mechanical energy. For fixed parameters, $\dot{V} = -[V'(\phi)]^2 \leq 0$. Linearizing at the two coordination modes gives

$$\lambda_0 = -a_H - 4b_H < 0, \quad \lambda_\pi = a_H - 4b_H. \quad (8.14)$$

Thus in-phase $\phi = 0$ remains locally stable, whereas anti-phase $\phi = \pi$ is stable only for $b_H/a_H > 1/4$. If the ratio decreases through $1/4$ as cycling frequency increases, anti-phase loses stability. A chosen frequency law is required before assigning a transition frequency in Hz. A forced second-order phase oscillator is a different model requiring its own derivation.

For $a_H = 1 \text{ s}^{-1}$, changing b_H from 0.5 to 0.2 s^{-1} changes λ_π from -1 to $+0.2 \text{ s}^{-1}$ while λ_0 remains negative. Exactly $\phi = \pi$ is an equilibrium even after losing stability: a numerical experiment must include a small perturbation or a declared noise process to observe departure. Near the transition, slow recovery and noise-induced switching can complicate identification of the deterministic threshold.

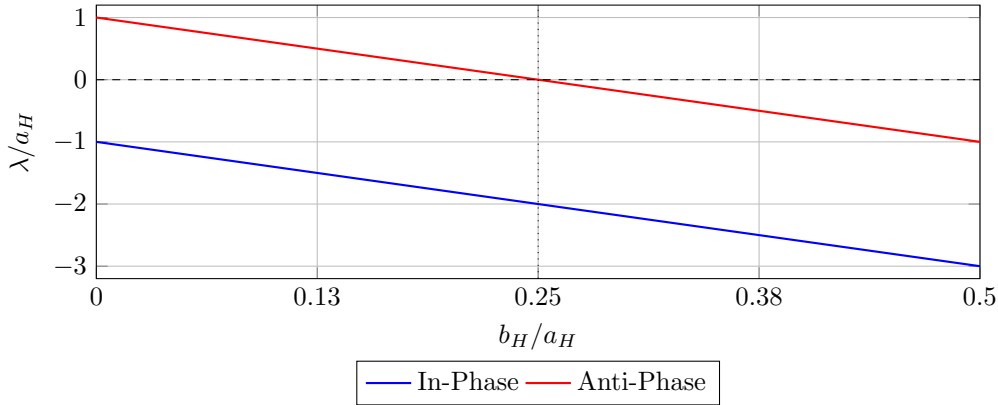


Figure 8.2: Local Phase Stability in the HKB Reduction. Negative Rates Indicate Recovery; Anti-Phase Loses Stability as the Ratio Falls Below One Quarter.

The reduced model explains an experimental coordination transition without an explicit discrete mode-selection command in its equations. It does not show that neurons are absent, that the brain never predicts trajectories or that golf timing follows the same phase law. In a swing, a proposed collective variable needs a measurement definition, fitted dynamics and tests under perturbations and altered tempo. A recognizable pattern is a starting point for that work, not its causal conclusion.

8.4 Synergies, Task Geometry and Control Authority

8.4.1 What a Low-Dimensional Fit Establishes

A spatial muscle-synergy model approximates an EMG or activation matrix $U \in \mathbb{R}^{p \times N}$ by WC , with $W \in \mathbb{R}^{p \times k}$ and $C \in \mathbb{R}^{k \times N}$. Nonnegative factorization usually imposes $W, C \geq 0$.

Specify preprocessing, normalization, muscles, tasks and rank selection; EMG is not a direct measurement of muscle force.

$$\text{VAF}_{\text{uncentered}} = 100 \left(1 - \frac{\|U - WC\|_F^2}{\|U\|_F^2} \right) \%, \quad \|U\|_F > 0. \quad (8.15)$$

The percentage multiplies the entire bracket. A centered variance definition uses a different denominator and must be named. High reconstruction quality is a property of the dataset and fit, not proof of a neural module or a universal number of synergies.

Kutch and Valero-Cuevas supplied counterexamples in which biomechanics produces low-dimensional activation patterns without fixed neural grouping [26]. Such alternatives do not establish that neural synergies never exist. They show why a causal claim needs task variation, perturbation, held-out prediction and comparison with independently controlled muscles. A useful robot policy may intentionally restrict its inputs to synergies even when the corresponding human mechanism remains unresolved.

8.4.2 A Task Level Set and Its Tangent Space

For a smooth task map $y = h(q)$ and target y_* , the exact task-preserving set is

$$\mathcal{M}_{y_*} = \{q : h(q) = y_*\}. \quad (8.16)$$

At a regular point $\bar{q}$ with rank- r Jacobian $J = Dh(\bar{q})$, its tangent space is $\ker J$ and has dimension $n-r$. The local uncontrolled-manifold analysis uses this linear approximation [35]. A finite step in a tangent direction generally leaves the curved level set:

$$h(\bar{q} + \delta q) - h(\bar{q}) = J\delta q + \frac{1}{2}D^2h(\bar{q})[\delta q, \delta q] + O(\|\delta q\|^3). \quad (8.17)$$

For $h(q) = q_1^2 + q_2^2$, $\bar{q} = (1, 0)$ and $\delta q = (0, 0.1)$, $J\delta q = 0$ but the task value increases from 1 to 1.01. A singular point requires a separate analysis; a changing Jacobian rank invalidates one fixed dimension formula across the whole region.

Using a declared Euclidean metric, let $P_N = I - J^\dagger J$ and $P_R = J^\dagger J$. For joint-deviation covariance Σ , dimension-normalized variances are

$$V_{\text{UCM}} = \frac{\text{tr}(P_N \Sigma)}{n - r}, \quad V_{\text{ORT}} = \frac{\text{tr}(P_R \Sigma)}{r}, \quad 0 < r < n. \quad (8.18)$$

Coordinates mixing lengths and angles require meaningful scaling or another metric. For isotropic noise $\Sigma = \sigma^2 I$, both normalized variances equal σ^2 ; redundancy alone does not give a ratio greater than one. Repeated trials also need alignment, a task definition and a treatment of measurement noise. More variance in the tangent directions supports a particular task-stabilization hypothesis; it does not establish that these directions are neurologically uncontrolled or irrelevant to every goal.

In golf, a null direction for hand position may change face orientation, shaft strain, grip loading or a later collision. The chosen output should therefore reflect the actual question. Kinematic task-preserving variations and muscle activation synergies live in different spaces; relating them requires muscle geometry and dynamics rather than identifying the two subspaces by name.

8.4.3 Instantaneous Rank Is Not Controllability

If an input policy imposes $u = Wc$ in $\dot{x} = f(x) + G(x)u$, the instantaneous input map is GW , with rank at most k . Nonnegative and bounded amplitudes further limit the feasible directions to a cone or bounded set. This does not mean that the other state directions are uncontrollable over time.

For the double integrator $\dot{q} = v$, $\dot{v} = u$, the instantaneous input matrix is $B = (0, 1)^T$, but

$$A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad [B, AB] = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (8.19)$$

has rank two. Drift transports the input's effect into position. Along a general nominal motion, use the linearized pair $A(t), B(t)$ and, where appropriate, its finite-horizon Gramian. Nonlinear accessibility, finite-time reachability and controllability under bounds are distinct additional questions.

Nor does a direction invisible in the current output remain unobservable. For the same double integrator with output $y = q$, velocity is initially in $\ker C$, $C = (1, 0)$, yet $[C^T, (CA)^T]^T$ has rank two. Position measured over time reveals velocity in the ideal model. There is no theorem that discarded synergy directions must coincide with an unobservable UCM. The practical test is whether the restricted policy can meet the chosen task and disturbance tolerances within its available time and force limits.

8.5 Impedance and Co-Contraction

8.5.1 Force, Displacement and Velocity Ports

For a local scalar perturbation model around a fixed operating point,

$$M\delta\ddot{x} + D\delta\dot{x} + K\delta x = \delta F, \quad M, D, K > 0, \quad (8.20)$$

the zero-initial-condition Laplace relations are

$$S(s) = \frac{\delta F(s)}{\delta X(s)} = Ms^2 + Ds + K, \quad (8.21)$$

$$Z(s) = \frac{\delta F(s)}{\delta V(s)} = Ms + D + K/s, \quad (8.22)$$

$$H(s) = \frac{\delta X(s)}{\delta F(s)} = \frac{1}{Ms^2 + Ds + K}. \quad (8.23)$$

S is dynamic stiffness (N/m); Z is mechanical impedance (Ns/m); H is displacement compliance. Some robotics literature uses “impedance control” for enforcing the complete mass–damping–stiffness relation. Naming that strategy does not remove the distinction between the transfer functions and their units. Multi-axis impedance needs matrices, a frame and a power-consistent port.

The natural frequency is $\omega_n = \sqrt{K/M}$ and damping ratio $\zeta = D/(2\sqrt{MK})$. Increasing K at fixed D raises ω_n but lowers ζ . For the chosen $M = 1$ kg, $D = 10$ Ns/m and $K = 100$ N/m, $\omega_n = 10$ rad/s and $\zeta = 0.5$. Increasing K to 400 N/m doubles the natural frequency and halves the damping ratio. A faster natural time scale does not guarantee less overshoot, smaller contact forces or better performance.

8.5.2 How Muscle Forces Contribute to Joint Stiffness

At fixed activation and internal-state assumptions, take $\tau(q) = -\sum_i F_i(l_i(q))\nabla l_i(q)$. Linearized restoring stiffness is

$$K_q = -\frac{\partial \tau}{\partial q} = \sum_i [k_i \nabla l_i \nabla l_i^T + F_i \nabla^2 l_i], \quad k_i = \frac{\partial F_i}{\partial l_i}. \quad (8.24)$$

The first term is material stiffness transmitted through moment arms; the second is geometric stiffness from loaded, changing geometry. Muscle-tendon series compliance changes the effective k_i . Coupled muscle dynamics, reflexes and active feedback require a frequency-dependent extension.

In a chosen one-joint model with constant opposing moment arms of 0.03 m, equal forces of 100 N produce zero net torque. If both effective longitudinal stiffnesses are 10,000 N/m, the joint stiffness is $2(0.03)^2(10,000) = 18$ N m/rad. The antagonists carry real load even though their net torque cancels. Equal activation need not produce equal opposing torques when muscle sizes, lengths or moment arms differ.

Hogan's analysis and forearm-posture experiment support a role for antagonist coactivation in impedance modulation [18]. They do not establish a universal stiffness proportional to target uncertainty. Target uncertainty, external force disturbances, observation noise and activation-dependent noise enter different parts of a control problem. Increasing stiffness can resist a force disturbance while increasing the force caused by a position error against a stiff environment. The disturbance and the scored outcome must be named.

8.5.3 An Explicit Optimization Example

For a quasistatic scalar model $e = w/K$ with zero-mean force disturbance of variance σ_F^2 , suppose a designer chooses the surrogate

$$J(K) = \frac{\sigma_F^2}{K^2} + \rho K^2, \quad 0 < K_{\min} \leq K \leq K_{\max}, \quad \rho > 0. \quad (8.25)$$

Weights carry the units needed to compare the terms. The first term is expected squared displacement; the second is an assumed stiffness penalty, not measured metabolic energy. Before applying bounds, differentiation gives

$$K_0 = (\sigma_F^2/\rho)^{1/4}, \quad K^* = \text{clip}(K_0, K_{\min}, K_{\max}). \quad (8.26)$$

For $\sigma_F = 10$ N, $\rho = 10^{-6}$ m⁴/N² and bounds 20–150 N/m, $K^* = 100$ N/m. Four times the force standard deviation would make the unconstrained optimum 200 N/m and the bounded optimum 150 N/m. Maximum feasible stiffness can therefore be optimal for a stated task. Changing the penalty, disturbance spectrum or dynamics changes the result; there is no general linear “signal-to-noise” stiffness law.

For the stable compliance H above, $\|H\|_\infty = \sup_\omega |H(i\omega)|$ measures a frequency-domain gain and the induced L_2 gain under the usual zero-initial-condition assumptions. It does not guarantee bounded output for arbitrary unlimited disturbances or robustness to unspecified model uncertainty. For this scalar system,

$$\|H\|_\infty = \begin{cases} 1/K, & \zeta \geq 1/\sqrt{2}, \\ 1/[2\zeta K \sqrt{1-\zeta^2}], & 0 < \zeta < 1/\sqrt{2}. \end{cases} \quad (8.27)$$

The dynamic-stiffness polynomial S instead grows without bound at high frequency; it is not the stable proper disturbance-to-displacement transfer whose finite norm was intended. In a real controller, sensing, filters, delay, contact and actuator dynamics determine the relevant closed-loop transfer.

8.5.4 Mechanical Work and Metabolic Cost

The local passive model stores $E = \frac{1}{2}M\delta\dot{x}^2 + \frac{1}{2}K\delta x^2$ and, for constant M, K , obeys $\dot{E} = \delta F\delta\dot{x} - D\delta\dot{x}^2$. If stiffness is varied, an additional $\frac{1}{2}\dot{K}\delta x^2$ term enters the energy balance. An actively changed spring parameter can supply energy; passivity cannot be inferred from positive instantaneous K alone.

Physiological metabolic cost also depends on activation, contraction history, force and velocity, tissue properties and the selected metabolic model. It is not generally proportional to $\int K dt$, net joint work, or squared inverse-dynamics torque. A golfer can have low net wrist torque while opposing muscles are loaded. A robot can hold a load with zero joint work while its motor dissipates electrical energy. Report the measured or modeled cost actually used; mechanical cancellation does not make physiological effort disappear.

8.6 Pre-Stress and Stability of Interacting Subsystems

8.6.1 What a Tensegrity Analogy Can Contribute

Pre-stress means an internal load state present before an additional perturbation. A self-stress in a free structural model balances internally without external nodal load. More generally, a loaded structure is linearized around an equilibrium that includes its external loads. Neither definition says that “internal forces exceed external forces”; these are different force collections, not one scalar comparison.

For a declared elastic potential $U(q)$, the constrained tangent stiffness comes from the appropriate second variation of total potential at equilibrium. Material and geometric contributions both matter, as in the preceding muscle-path formula. Preloading can stiffen some directions and soften others, including buckling. Equilibrium force balance alone does not prove stability.

Tensegrity models can help formulate questions about distributed load paths. Anatomical bones also experience bending, shear and joint-contact forces, while muscles, tendons and fascia have distributed geometry and constitutive behavior. A cell-scale or ideal strut-cable model is not by itself validation of a whole-body architecture. Rigid-body multibody models can already include muscle forces, compliance and loaded constraints; these modeling approaches need not be treated as mutually exclusive.

For golf, useful questions concern the force path from ground through legs, trunk, arms and grip, and the resulting tangent response under a defined perturbation. Muscle tone supplies force and may change that response; it is not separate from force generation. A proposed fascia-mediated mechanism must predict an observable effect beyond what the competing joint/contact model already explains. A humanoid comparison likewise needs a specified structure, actuation and contact model, rather than an anatomical analogy alone.

8.6.2 Why Stable Parts Can Form an Unstable Whole

Organizing analysis into tissue, intermuscular and task levels is useful. Their time scales overlap, however, and their couplings must remain in the equations. A spinal feedback loop is a CNS process, and faster mechanics does not eliminate sensing, prediction or adaptation. Separate stability does not guarantee that the sum of the levels' Lyapunov functions is a composite certificate: coupling terms must also be controlled.

Example 8.2. An Unstable Interconnection of Stable Scalar Parts

Consider

$$\dot{z}_1 = -z_1 + 2z_2, \quad \dot{z}_2 = -z_2 + 2z_1. \quad (8.28)$$

Each isolated equation decays when the other state is fixed at zero. The coupled matrix has eigenvalues 1 and -3 . For $V = (z_1^2 + z_2^2)/2$, $\dot{V} = -z_1^2 - z_2^2 + 4z_1z_2$ is positive at $z_1 = z_2 \neq 0$. Separate stability has not controlled the feedback interaction.

Theorem 8.1 (A Sufficient Composite Decay Condition). *Suppose positive-definite local functions V_i for a common equilibrium satisfy, on a specified neighborhood,*

$$\dot{V}_i \leq -\alpha_i \|z_i\|^2 + \sum_{j \neq i} \gamma_{ij} \|z_i\| \|z_j\|, \quad \alpha_i > 0, \quad \gamma_{ij} \geq 0.$$

If weights $w_i > 0$ make the symmetric matrix with entries

$$S_{ii} = w_i \alpha_i, \quad S_{ij} = -\frac{1}{2}(w_i \gamma_{ij} + w_j \gamma_{ji})$$

positive definite, then $V = \sum_i w_i V_i$ has strictly negative derivative away from the equilibrium in that neighborhood. With the usual regularity and local positive-definiteness bounds, it proves local asymptotic stability.

Proof. Set $r_i = \|z_i\|$. Summing the derivative bounds gives $\dot{V} \leq -r^T S r \leq -\lambda_{\min}(S) \|r\|^2$. Small enough sublevel sets remain inside the neighborhood; the Lyapunov criterion then applies. A global conclusion would require global hypotheses. $\square$

For three scalar levels with diagonal decay one and symmetric neighbor coupling 0.2, the decay matrix has smallest eigenvalue $1 - 0.2\sqrt{2} \simeq 0.7172 > 0$. This chosen example passes the condition. It is not an estimate of a nervous system's coupling strength. Other systems may need input-to-state stability, small-gain, passivity or singular-perturbation analysis. For contact switches, also check reset-induced changes in the certificate; continuous decay between events is not a complete hybrid stability proof.

The resulting research claim is specific: suitable coupling and feedback can make distributed stabilization work. A hierarchy is an analysis strategy, not automatic protection from dimensionality, delay or instability.

8.7 Building and Testing a Coupled Movement Model

8.7.1 Choose the Model for the Question

Inverse dynamics recovers the net generalized loads consistent with specified kinematics, inertia and external loads. It does not inherently assume that muscle activation is instantaneous or that tendons are rigid: those assumptions belong to a chosen actuator model.

Recovering a joint moment does not identify the individual muscle forces producing it. Ballistic movements can be particularly sensitive to activation history, elasticity and contact timing; speed alone does not justify ignoring those states.

A practical model comparison starts with a rigid-body net-load model, then adds only the states needed to answer the question. A muscle-driven model can add activation, fiber and tendon dynamics; a humanoid actuator model can add motor current, transmission compliance and control delay. Flexible shafts and grips require corresponding deformation states. Both versions should retain the same task, initial-condition specification and external-load convention.

Check equilibrium, power balance, force bounds, constraint residuals and numerical convergence before interpreting differences. A more elaborate model can fit more observations while being less identifiable. Adding springs or feedback does not guarantee useful attractors, favorable energy transfer or better predictions. Compare the simplest competing models against held-out data.

8.7.2 Identification Requires More Than a Motion Fit

Let measurements satisfy $y_k = \mathcal{H}(x(t_k), \theta) + \epsilon_k$, with parameters θ . The local sensitivity matrix

$$\mathcal{S}_{kj} = \frac{\partial y_k}{\partial \theta_j} \quad (8.29)$$

reveals combinations distinguishable in the selected experiment. When columns are nearly dependent, different parameters can compensate for one another. Measurement covariance, parameter scaling and model discrepancy all affect the interpretation. A successful optimizer does not establish physiological uniqueness or remove uncertainty.

Joint motion and ground reactions constrain net mechanics. EMG adds timing and activation-related evidence with its own calibration limitations. Ultrasound or other tissue measurements can constrain fiber or tendon behavior when the measurement is appropriate. Perturbation experiments can separate stiffness, damping and delayed feedback more effectively than repeated unperturbed motion. No fixed parameter count guarantees identifiability, and imaging is not uniformly invasive. Specify what each measurement adds and what remains inferred.

For learning and fatigue, parameters or policies may change across trials. Separate within-trial state estimation from across-trial adaptation. A model that refits each swing independently can conceal a failure to predict how a changed grip, tempo, load or support condition changes the response.

8.7.3 A Golf and Humanoid Experiment Ladder

1. **Define the delivered state.** Record the clubhead or tool frame, reference point, contact event and task tolerance. State whether the objective is face control, speed, balance, disturbance recovery or a combination. Joint angles alone do not specify all of these outcomes.
2. **Define comparable initial conditions.** Match or estimate posture, velocity, activation and elastic state. Distinguish a new command from a changed starting state. A zero-excitation continuation preserves the existing activation and strain history; it is not an instruction to make the person instantaneously relaxed.

3. **Apply a declared disturbance.** Vary tool inertia, support, grip loading or a measured external perturbation within an appropriate protocol. Specify the time and direction. Disturbances applied before and near impact probe different available correction horizons.
4. **Measure both task and internal response.** Compare delivered state, phase error, hand/ground loads and relevant activation or strain evidence. A successful endpoint with larger opposing forces may trade robustness for effort. Lower joint torque alone cannot decide whether that trade occurred.
5. **Compare mechanisms.** Test a net-load model, an augmented tissue or actuator model, and alternative feedback policies using the same scored data. Report prediction intervals and failures, including cases in which added compliance or reduced input dimension worsens the task.

These comparisons expose how the components are intertwined. Posture changes moment arms and inertia; activation and strain change force and local response; contact changes feasible reactions; those changes alter the useful feedback directions and remaining time. Earlier work can become later kinetic or elastic energy, while a change that improves speed may impair force control or loading. The joint explanation is a sequence of measurable consequences, not a claim that mechanics replaces neural control.

8.7.4 What Would Count Against the Proposed Explanation?

A reflex-based explanation weakens if the estimated intrinsic response is too small or has the wrong sign to explain early recovery. A tendon-recoil explanation weakens if the bounded recoverable strain energy is insufficient or is released at the wrong time. A fixed-synergy explanation weakens if it fails to predict withheld task directions or perturbations that an independently actuated model predicts. An attractor explanation weakens if the supposed recovery disappears after phase alignment or depends on unmodeled powered feedback. A hierarchical stability claim fails if cross-coupling or contact resets defeat its proposed certificate.

These are model-specific tests, not reasons to reject biological coordination as a whole. They make the useful positive argument stronger: a successful model explains both why a pattern works in its operating region and why it stops working when the relevant conditions change.

8.8 Connection to the Series

Geometry supplies configurations, tangent spaces and moment arms. Multibody dynamics translates applied loads into motion. Muscle and actuator models describe how command history becomes load and impedance. Feedback, prediction and learning shape behavior within those physical limits. Contact and impact determine how the delivered state becomes task outcome. Each layer adds an assumption and a corresponding opportunity for validation.

The neighboring treatments of passive control, central pattern generation, motor learning and robotics should be read with this separation in mind. Distributed mechanics and neural computation can cooperate; a low-dimensional description can be useful without uniquely

identifying a neural implementation. The strongest golf-swing argument specifies the state, intervention, constraints, time horizon and observable consequence. The same discipline supports a humanoid-tool model while preserving the differences in its actuators and sensing.

Exercises

1. **Force–Velocity and Series Equilibrium.** Verify the values and one-sided derivatives of the stated f_V at zero. Use the declared force–length and tension-only tendon laws with zero pennation to solve a prescribed length-history example. Increasing fiber length is lengthening, not shortening. Report how tendon compliance and activation time scale separately affect the result; do not prescribe inconsistent fiber and tendon lengths independently.
2. **Stored Energy and Zero Excitation.** Reproduce the 4 J tendon example and its ideal average recoil power. Then use $a(t_0) = 0.8$ and a chosen $\tau_a = 0.04$ s to calculate activation 0.04 s after zeroing excitation. Explain why the resulting $0.8/e \simeq 0.2943$ need not produce zero active force. Close the energy balance when losses are added.
3. **Orbit and Trajectory Stability.** Simulate the radial/phase example from two nearby initial phases and radii. Plot distance to the orbit and time-matched distance between trajectories. Explain which measure is appropriate for a periodic rhythm and which additional timing information a collision task requires.
4. **HKB Transition.** Use the first-order phase equation with $a_H = 1$ and decrease b_H through 0.25. Start near, rather than exactly at, π . Compare the observed recovery rate with $a_H - 4b_H$ and test sensitivity to perturbation size and parameter-ramp speed. A transition frequency cannot be reported until a frequency-to-parameter relationship has been specified.
5. **Uncontrolled-Manifold Null Model.** At a regular configuration of a three-link planar arm, compute a rank-two position Jacobian and the dimension-normalized covariance projections. First use isotropic joint noise: the population ratio is one, not automatically greater than one. Then impose a larger covariance in the tangent direction. Compare linearized task variation with the actual nonlinear forward kinematics as perturbation size increases. Repeat with orientation included in the task.
6. **Impedance and Coupling Trade-Offs.** Reproduce the bounded stiffness optimum, then compare disturbance compliance and contact force under a prescribed position error. Verify the two-level unstable interconnection and the three-level sufficient decay condition. Finally introduce a delay or reset and determine which earlier certificate must be reconsidered. A favorable result from one objective is not a universal ranking of stiff and compliant strategies.

Chapter 9

Central Pattern Generators

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Rhythm is the most ancient form of motor control.

9.1 CPGs in Biological Motor Control

Central pattern generators are neural circuits that produce rhythmic motor output without rhythmic sensory input or supraspinal commands. Evidence for CPGs:

- Isolated lamprey spinal cord generates swimming patterns [8]
- Deafferented cats produce coordinated stepping on treadmills [13]
- Songbird pre-motor nuclei show rhythmic activity in vitro [39]

CPGs underlie locomotion, respiration, chewing, and other rhythmic behaviors.

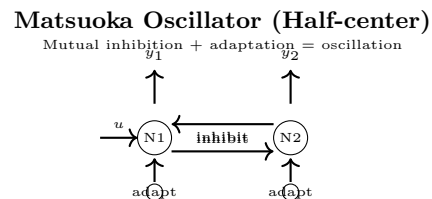


Figure 9.1: Matsuoka oscillator CPG circuit: mutual inhibition between neurons with adaptation variables creates an alternating rhythm. Tonic input frequency-modulates the oscillation without losing periodicity.

9.2 The Matsuoka Oscillator

The most widely used CPG model in robotics is the Matsuoka oscillator [30]:

$$\tau_1 \dot{x}_i = -x_i - \beta v_i - \sum_{j \neq i} w_{ij} y_j + u_i + s_i, \quad (9.1)$$

$$\tau_2 \dot{v}_i = -v_i + y_i, \quad (9.2)$$

$$y_i = \max(0, x_i), \quad (9.3)$$

where x_i is the membrane potential, v_i is the adaptation variable, y_i is the output, w_{ij} are inhibitory coupling weights, u_i is the tonic input, and s_i is the sensory feedback.

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 class MatsuokaOscillator:
5     """Two-neuron Matsuoka oscillator for CPG-based control."""
6
7     def __init__(
8         self,
9         tau1: float = 0.1,
10        tau2: float = 0.3,
11        beta: float = 2.5,
12        w_mutual: float = 2.0,
13        tonic_input: float = 1.0
14    ):
15        self.tau1 = tau1
16        self.tau2 = tau2
17        self.beta = beta
18        self.w = w_mutual
19        self.u = tonic_input
20
21    def dynamics(self, t: float, state: np.ndarray) -> np.ndarray:
22        x1, v1, x2, v2 = state
23        y1 = max(0.0, x1)
24        y2 = max(0.0, x2)
25
26        dx1 = (-x1 - self.beta * v1 - self.w * y2 + self.u) / self.tau1
27        dv1 = (-v1 + y1) / self.tau2
28        dx2 = (-x2 - self.beta * v2 - self.w * y1 + self.u) / self.tau1
29        dv2 = (-v2 + y2) / self.tau2
30
31        return np.array([dx1, dv1, dx2, dv2])
32
33    def simulate(self, t_end: float = 5.0,
34                dt: float = 0.001) -> tuple[np.ndarray, np.ndarray]:
35        state0 = np.array([0.1, 0.0, -0.1, 0.0])
36        t_eval = np.arange(0, t_end, dt)
37        sol = solve_ivp(self.dynamics, [0, t_end], state0,
38                        t_eval=t_eval, method='RK45')
39
40        # Extract outputs
41        y1 = np.maximum(0, sol.y[0])
42        y2 = np.maximum(0, sol.y[2])
43        output = y1 - y2 # Net output
44
45        return sol.t, output
46

```



```

47
48 # Simulate and print summary
49 cpg = MatsuokaOscillator(tau1=0.1, tau2=0.3)
50 t, output = cpg.simulate(t_end=5.0)
51
52 # Estimate frequency from zero crossings
53 crossings = np.where(np.diff(np.sign(output)))[0]
54 if len(crossings) >= 4:
55     periods = np.diff(t[crossings[:2]])
56     freq = 1.0 / np.mean(periods)
57     print(f"CPG frequency: {freq:.2f} Hz")
58     print(f"CPG amplitude: {np.max(np.abs(output[len(output)//2])):.3f}")

```

Listing 9.1: Matsuoka oscillator CPG implementation.

9.3 Phase Coupling and Inter-Limb Coordination

Multiple CPGs can be coupled to produce coordinated multi-limb movement:

$$\dot{\phi}_i = \omega_i + \sum_j K_{ij} \sin(\phi_j - \phi_i - \psi_{ij}), \quad (9.4)$$

where ϕ_i is the phase of the i -th oscillator, ω_i is its natural frequency, K_{ij} is the coupling strength, and ψ_{ij} is the desired phase relationship.

For quadruped locomotion:

- Walk: $\psi = (0, \pi, \pi/2, 3\pi/2)$ (diagonal pairs alternating)
- Trot: $\psi = (0, \pi, \pi, 0)$ (diagonal pairs synchronized)
- Gallop: $\psi = (0, 0, \pi, \pi)$ (front-back alternating)

Exercises

1. **CPG Tuning.** Vary τ_1/τ_2 ratio and plot the resulting frequency and duty cycle. How does the ratio control the waveform shape?
2. **Phase Coupling.** Implement 4 coupled oscillators and produce walk, trot, and gallop patterns by varying ψ_{ij} .
3. **(Programming.)** Use the Matsuoka CPG to drive a simulated bipedal walker. Tune the CPG parameters for stable walking.

Chapter 10

Motor Learning and Adaptation

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Practice does not make perfect.
Practice makes permanent. Only
perfect practice makes perfect.

10.1 Three Learning Systems

Motor learning involves three distinct neural systems with different timescales and mechanisms:

1. **Error-based learning** (cerebellum): uses sensory prediction errors (climbing fiber signals) to update cerebellar forward models via LTD [29, 24]. Trial-by-trial timescale (minutes to hours). It drives adaptation both to altered limb *dynamics* [37] and to an altered visuomotor *mapping* [25] — and those two are acquired independently of one another, so they are worth keeping distinct rather than filing together as “adaptation”.
2. **Reinforcement learning** (basal ganglia): uses dopamine reward signals to strengthen successful motor patterns [36]. Slower than error-based learning (hours to days). Enables discovery of novel solutions not implicit in the task structure.
3. **Consolidation and structural learning** (cortical): repetition of a movement biases future movements toward repeated patterns [9]. Timescale of hours to days; explains habit formation. Involves gradual changes in M1 tuning and synaptic weights.

10.2 Visuomotor Adaptation

The classic paradigm for studying error-based learning [25]:

```
1 import numpy as np
2
3 def simulate_visuomotor_adaptation(
4     n_baseline: int = 50,
```

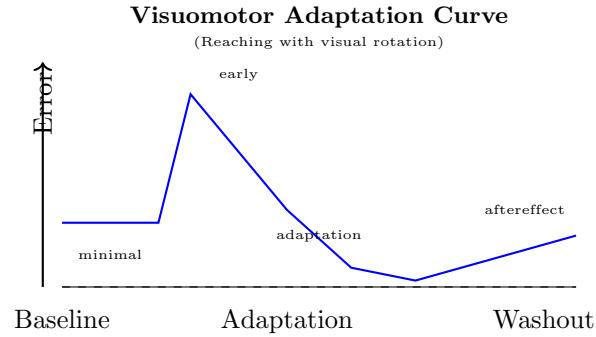


Figure 10.1: Characteristic visuomotor adaptation profile [25]: rapid error reduction (first 20 trials), gradual asymptote, persistent aftereffect demonstrating learning consolidation.

```

5     n_adaptation: int = 200,
6     n_washout: int = 100,
7     perturbation_deg: float = 30.0,
8     learning_rate: float = 0.05,
9     retention: float = 0.98
10 ) -> dict:
11     """Simulate visuomotor rotation adaptation.
12
13     A simple state-space model of adaptation:
14      $x_{k+1} = \text{retention} * x_k + \text{learning\_rate} * e_k$ 
15
16     Parameters
17     -----
18     n_baseline : int
19         Baseline trials (no perturbation).
20     n_adaptation : int
21         Adaptation trials (perturbation on).
22     n_washout : int
23         Post-adaptation trials (perturbation off).
24     perturbation_deg : float
25         Rotation perturbation in degrees.
26     learning_rate : float
27         Rate of error-based learning.
28     retention : float
29         Memory retention between trials.
30
31     Returns
32     -----
33     dict
34         Keys: 'trial', 'reaching_error', 'internal_state',
35              'phase' (labels for each trial)
36
37     """
38     n_total = n_baseline + n_adaptation + n_washout
39     p = np.radians(perturbation_deg)
40
41     # Internal state (learned compensation)
42     x = 0.0
43     errors = np.zeros(n_total)
44     states = np.zeros(n_total)
45     phases = []

```

```

46     for trial in range(n_total):
47         if trial < n_baseline:
48             perturbation = 0.0
49             phases.append('baseline')
50         elif trial < n_baseline + n_adaptation:
51             perturbation = p
52             phases.append('adaptation')
53         else:
54             perturbation = 0.0
55             phases.append('washout')
56
57         # Reaching error = perturbation - compensation
58         error = perturbation - x
59         errors[trial] = np.degrees(error)
60         states[trial] = np.degrees(x)
61
62         # Update internal state
63         x = retention * x + learning_rate * error
64
65     return {
66         'trial': np.arange(n_total),
67         'reaching_error': errors,
68         'internal_state': states,
69         'phase': phases
70     }
71
72
73 result = simulate Visuomotor adaptation()
74 print("Visuomotor Adaptation Summary:")
75 print(f" Baseline error: {np.mean(np.abs(result['reaching_error'][:50])):.1f} deg")
76 print(f" Final adaptation error: "
77       f"{np.abs(result['reaching_error'][-1]):.1f} deg")
78 print(f" Aftereffect: {result['reaching_error'][-1]:.1f} deg")

```

Listing 10.1: Visuomotor adaptation simulation.

10.3 Savings and Interference

10.3.1 Savings

Re-learning a previously learned adaptation is faster than initial learning, even after complete washout [20] — an observation that predates motor control, going back to Ebbinghaus's 1885 work on memory. This savings effect suggests that motor memories are not erased but suppressed, with additional learning circuits retaining implicit information about the learning dynamics.

10.3.2 Interference and Dual-Rate Learning

Learning opposite perturbations (A then B) produces anterograde interference: learning B slows retention of A (Li and Xie, 2000). This is explained by dual-rate learning models: a fast process learns trial-by-trial; a slow process consolidates. Interference occurs because the fast process is task-specific while the slow process is more general.

10.4 Structural Learning and Meta-Learning

Over many adaptation experiences, subjects develop the ability to adapt faster—they learn to learn (context-dependent learning). Smith et al. (1998) showed that prior exposure to one visuomotor rotation task accelerates learning of a novel rotation. This involves learning the structure of the task space itself, not just individual parameter values. Brain areas like dorsolateral prefrontal cortex appear to encode such meta-learning (Gershman et al., 2017).

Exercises

1. **State-Space Model.** Using the provided code, explore how learning rate and retention affect adaptation speed and aftereffect size.
2. **Dual-Rate Model.** Implement a dual-rate model (fast + slow process) and show that it can explain savings.
3. **(Programming.)** Simulate interference by adapting to $+30^\circ$ then -30° . Plot the adaptation curves and aftereffects.

Chapter 11

Computational Models of Motor Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

A theory of motor control must ultimately be a theory of computation: what is computed, how, and why.

Table 11.1: Comparison of Computational Motor Control Theories.

Theory	Learning Rule	Variability	Key Prediction
OFC (Todorov, 2004)	Error feedback	Tolerate task-irrelevant	Bell-shaped velocity profiles
Active Inference (Friston, 2010)	Minimize free energy	Prediction error-driven	Sensorimotor uncertainty shapes movement
Internal Model (Kawato, 1999)	Forward model learning (climbing fiber error)	Exploration + exploitation	Gradual visuomotor adaptation
Synergy Control (d’Avella, 2005)	Reduced-dim learning (NMF, ICA)	Synergy-space variability allowed	Movement flexibility without dimensionality

11.1 Optimal Feedback Control

The dominant computational theory of motor control is optimal feedback control (OFC), formulated by Todorov and Jordan [41]. OFC explains motor variability patterns (minimum intervention principle), predicts smooth velocity profiles, and unifies cerebellar learning with optimal control:

Definition 11.1. The OFC Framework

The nervous system controls movement by solving, in real-time, the stochastic optimal control problem:

$$\min_{\mathbf{u}(\cdot)} \mathbb{E} \left[\phi(\mathbf{x}_T) + \int_0^T \ell(\mathbf{x}_t, \mathbf{u}_t) dt \right], \quad (11.1)$$

subject to:

$$d\mathbf{x} = f(\mathbf{x}, \mathbf{u}) dt + G(\mathbf{x}) d\mathbf{w}, \quad (11.2)$$

where $d\mathbf{w}$ is Brownian noise representing signal-dependent motor noise.

Equation (11.1) is a *stochastic* optimal control problem, and the distinction matters. Because the dynamics carry a Brownian term, the solution is characterised by a Hamilton–Jacobi–Bellman equation for the value function, not by the Pontryagin maximum principle applied along a single trajectory. The deterministic geometric control theory this series draws on elsewhere—Agrachev & Sachkov [1]—treats smooth ordinary differential equations with no noise term and develops optimality through Pontryagin’s principle; it does not cover the problem above.

The reason a stochastic formulation is needed at all is that motor noise scales with the magnitude of the control signal, so it cannot be treated as a small additive disturbance around a nominal plan [15]. That single fact is what makes the minimum intervention principle below a prediction rather than a preference.

11.1.1 The Minimum Intervention Principle

OFC naturally explains why the nervous system tolerates task-irrelevant variability (the UCM from Chapter 1): the optimal controller only corrects deviations that affect the cost function. Task-irrelevant deviations are “free” and need not be corrected.

```

1 import numpy as np
2 from scipy.linalg import solve_continuous_are
3
4 def lqr_ofc(
5     A: np.ndarray,
6     B: np.ndarray,
7     Q: np.ndarray,
8     R: np.ndarray,
9     state_noise_cov: np.ndarray | None = None
10 ) -> tuple[np.ndarray, np.ndarray]:
11     """Compute LQR optimal feedback gains.
12
13     This implements the infinite-horizon OFC solution.
14
15     Parameters
16     -----
17     A : np.ndarray, shape (n, n)
18         State dynamics matrix.
19     B : np.ndarray, shape (n, m)
20         Input matrix.
21     Q : np.ndarray, shape (n, n)
22         State cost matrix.
23     R : np.ndarray, shape (m, m)
```

```

24         Control cost matrix.
25         state_noise_cov : np.ndarray, shape (n, n), optional
26             Process noise covariance.
27
28         Returns
29         -----
30         tuple
31             (K, P) where K is the optimal gain and P is the Riccati solution.
32         """
33         # Solve continuous algebraic Riccati equation
34         P = solve_continuous_are(A, B, Q, R)
35
36         # Optimal gain
37         K = np.linalg.solve(R, B.T @ P)
38
39         return K, P
40
41
42 # Example: 2D reaching with motor noise
43 # State: [x, y, dx, dy], Control: [Fx, Fy]
44 A = np.array([
45     [0, 0, 1, 0],
46     [0, 0, 0, 1],
47     [0, 0, -2, 0], # damping
48     [0, 0, 0, -2],
49 ])
50 B = np.array([
51     [0, 0],
52     [0, 0],
53     [1, 0],
54     [0, 1],
55 ])
56
57 # Task: reach to target with minimum effort
58 # High cost on position at target, low cost on path
59 Q = np.diag([10, 10, 1, 1]) # Position matters more than velocity
60 R = 0.01 * np.eye(2) # Low control cost
61
62 K, P = lqr_ofc(A, B, Q, R)
63 print("OFC gain matrix K:")
64 print(K)
65 print("\nMinimum intervention: gains are higher for")
66 print("task-relevant (position) than task-irrelevant dimensions")

```

Listing 11.1: Linear-quadratic optimal feedback controller.

11.2 Signal-Dependent Noise

A key feature of biological motor systems is that motor noise is **signal-dependent**: the noise variance scales with the control signal magnitude:

$$\mathbf{u} = \bar{\mathbf{u}} + \sigma(\bar{\mathbf{u}})\boldsymbol{\xi}, \quad \sigma(\bar{\mathbf{u}}) = k \|\bar{\mathbf{u}}\|, \quad (11.3)$$

where $\boldsymbol{\xi}$ is white Gaussian noise and k is the noise coefficient (empirically $k \approx 0.01$ – 0.05 in human reaching). This property has been measured extensively (Schmidt et al., 1979; Harris and Wolpert, 1998).

Under signal-dependent noise, the optimal trajectory minimizes integrated squared jerk (Flash and Hogan, 1985). This is a key prediction of OFC: the bell-shaped velocity profiles observed in human reaching emerge naturally as solutions to the signal-dependent noise minimization problem, not from any explicit minimization of jerk (Todorov, 2004).

11.3 Active Inference (Friston Framework)

Active inference (Friston, 2010; Friston et al., 2016) reframes motor control as inference: the agent minimizes prediction error by acting to make observations match its generative model. This is formalized through variational free energy minimization:

$$F = \mathbb{E}_q[\ln q(\mathbf{x}) - \ln p(\mathbf{x}, \mathbf{y})], \quad (11.4)$$

where F is the variational free energy, $q(\mathbf{x})$ is the approximate posterior belief, and $p(\mathbf{x}, \mathbf{y})$ is the generative model. The agent chooses actions to minimize F . This unifies action, perception, and learning under a single principle. While the experimental evidence is still developing, active inference correctly predicts many phenomena including the minimum intervention principle and why the brain is tolerant of task-irrelevant variability.

Exercises

1. **OFC Reaching.** Using the LQR code, simulate reaching movements to different targets. Show that task-irrelevant variability is tolerated.
2. **Signal-Dependent Noise.** Simulate reaching with and without signal-dependent noise. Plot the trajectory variability as a function of movement speed.
3. **(Programming.)** Implement the minimum-jerk model and the minimum-variance model. Show that both predict smooth bell-shaped velocity profiles.

Chapter 12

From Neural Architecture to Robot Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

If we truly understood how humans move, we could build robots that move like humans. We cannot. This tells us something.

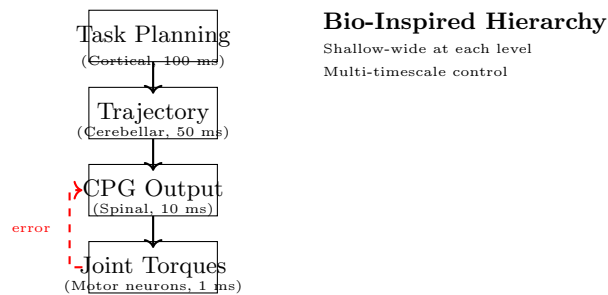


Figure 12.1: Hierarchical multi-timescale control architecture: each level shallow (1-2 layers) but wide (thousands of parallel channels), operating at increasing timescales and levels of abstraction.

12.1 Lessons From Biology for Robot Design

The four volumes of theory converge on practical design principles for robotic control systems:

Key Insight 12.1. Bio-Inspired Control Design Principles

1. **Exploit passive dynamics** (Vol II, Vol IV Ch 7): design the mechanical system so that natural dynamics do useful work. Use compliance, not rigidity.
2. **Use shallow-wide controllers** (Vol IV Ch 4): for real-time control, prefer 2–3 layer networks with thousands of neurons over deep networks with hundreds of layers.
3. **Employ synergy-based dimensionality reduction** (Vol IV Ch 4): define 4–8 actuation synergies and control in synergy space, not muscle space.
4. **Implement forward models** (Vol IV Ch 6): use the Smith predictor architecture to compensate for sensor/actuator delays.
5. **Design for contraction** (Vol I Ch 4): ensure the closed-loop system is contracting, guaranteeing stability and robustness.
6. **Optimize trajectories, not setpoints** (Vol II): design trajectory-centric controllers with funnel guarantees.
7. **Accept variability** (Vol IV Ch 1): allow task-irrelevant variability. Don't over-constrain the controller.

12.2 Cerebellar-Inspired Adaptive Control

The cerebellum's circuit architecture [29, 2, 24] directly suggests an adaptive control scheme:

- **Granule cells** (the bulk of some 69 billion cerebellar neurons [3]): create a compressed basis function representation of input state via mossy fiber–granule cell synapses
- **Purkinje cells** (~17 million): linearly combine granule cell outputs; synaptic weights change via LTD when climbing fiber error signals are present
- **Climbing fiber error** (inferior olive): carries sensory prediction error from brainstem nuclei; signals at ~1 Hz but with precise timing (10 ms resolution)

This maps directly to:

$$\mathbf{u}_{\text{adaptive}}(\mathbf{x}) = \hat{\mathbf{W}}^T \boldsymbol{\phi}(\mathbf{x}), \quad \dot{\hat{\mathbf{W}}} = -\Gamma \boldsymbol{\phi}(\mathbf{x}) \mathbf{e}^T, \quad (12.1)$$

where $\boldsymbol{\phi}(\mathbf{x})$ are radial basis functions (granule cell expansion), $\hat{\mathbf{W}}$ are adaptive weights (parallel fiber–Purkinje cell synapses), and $\mathbf{e}$ is the tracking error (climbing fiber signal).

12.3 CPG-Based Locomotion Controllers

Following Chapter 8, CPG-based locomotion controllers use coupled oscillators to generate walking/running patterns. This architecture is inspired by Grillner's [13] demonstration that deafferented spinal cords generate coordinated stepping. The CPG provides the *timing* and *synergy* structure; feedback provides *stability* and *responsiveness*:

```

1 import numpy as np
2
3 class BioInspiredLocomotionController:
4     """Hierarchical control architecture inspired by
5     biological motor control."""
6
7     def __init__(
8         self,
9         n_joints: int,
10        n_synergies: int = 4,
11        cpg_freq: float = 1.0
12    ):
13        self.n_joints = n_joints
14        self.n_synergies = n_synergies
15        self.cpg_freq = cpg_freq
16
17        # Synergy matrix (learned from motion data)
18        self.W = np.random.randn(n_joints, n_synergies) * 0.1
19
20        # CPG phases for each synergy
21        self.phases = np.linspace(
22            0, 2 * np.pi * (1 - 1/n_synergies), n_synergies
23        )
24
25        # Feedback gains (shallow: single layer)
26        self.K_fb = np.zeros((n_synergies, 2 * n_joints))
27
28    def cpg_output(self, t: float) -> np.ndarray:
29        """Generate rhythmic synergy activations from CPG."""
30        activations = np.array([
31            max(0.0, np.sin(2 * np.pi * self.cpg_freq * t + phi))
32            for phi in self.phases
33        ])
34        return activations
35
36    def feedforward(self, t: float) -> np.ndarray:
37        """Feedforward joint torques from CPG + synergies."""
38        c = self.cpg_output(t)
39        return self.W @ c
40
41    def feedback(self, state_error: np.ndarray) -> np.ndarray:
42        """Feedback correction (shallow: one layer)."""
43        c_fb = self.K_fb @ state_error
44        return self.W @ c_fb
45
46    def control(self, t: float,
47               state_error: np.ndarray) -> np.ndarray:
48        """Total control: feedforward (CPG) + feedback."""
49        return self.feedforward(t) + self.feedback(state_error)
50
51
52 # Example: 6-joint biped with 4 synergies
53 controller = BioInspiredLocomotionController(
54     n_joints=6, n_synergies=4, cpg_freq=1.2
55 )
56
57 # Generate one cycle of control signals
58 t = np.linspace(0, 1.0/1.2, 200)
59 torques = np.array([controller.feedforward(ti) for ti in t])
60

```

```

61 print(f"Bio-inspired controller: {controller.n_joints} joints, "
62       f"{controller.n_synergies} synergies")
63 print(f"Control dimensionality reduced from {controller.n_joints} "
64       f"to {controller.n_synergies}")
65 print(f"Peak torque magnitude: {np.max(np.abs(torques)):.3f}")

```

Listing 12.1: Bio-inspired locomotion controller architecture.

12.4 Whole-Body Control Architectures

For humanoid robots, the bio-inspired architecture suggests:

1. **Level 0 (Spinal)**: joint-level impedance control with reflex-like feedback (~ 1 ms loop)
2. **Level 1 (CPG)**: rhythmic pattern generation for locomotion (~ 10 ms loop)
3. **Level 2 (Cerebellar)**: adaptive feedforward based on forward model predictions (~ 50 ms loop)
4. **Level 3 (Cortical)**: trajectory planning and task-level control (~ 100 ms loop)

Each level is **shallow** (1–2 layers) but **wide** (many parallel channels), operating at progressively longer timescales.

12.5 Companion Software: Practical Implementation

Volume V provides the practical tools to implement and test these controllers using the pedagogical platform accompanying this textbook. The connection points:

- **MuJoCo**: test CPG-based locomotion controllers on musculoskeletal models
- **Drake**: implement trajectory optimization for trajectory-centric control
- **Pinocchio**: compute forward/inverse dynamics efficiently in real-time

Exercises

1. **Controller Architecture.** Using the bio-inspired controller code, vary the number of synergies and measure the resulting control signal dimensionality and smoothness.
2. **Cerebellar Adaptive Control.** Implement the cerebellar-inspired adaptive controller for a 2-DOF arm and show that it learns an inverse model over 50 trials.
3. **Hierarchical Design.** Design a 3-level hierarchical controller for a simulated biped, with impedance control at Level 0, CPG at Level 1, and trajectory correction at Level 2.
4. **(Programming.)** Implement the full bio-inspired controller for a MuJoCo walking model and benchmark against a standard PD controller.

Bibliography

- [1] Andrei A. Agrachev and Yuri L. Sachkov. *Control Theory from the Geometric Viewpoint*, volume 87 of *Encyclopaedia of Mathematical Sciences*. Springer, 2004.
- [2] J. S. Albus. A theory of cerebellar function. *Mathematical Biosciences*, 10(1-2):25–61, 1971.
- [3] Frederico A. C. Azevedo, Ludmila R. B. Carvalho, Lea T. Grinberg, José Marcelo Farfel, Renata E. L. Ferretti, Renata E. P. Leite, Wilson Jacob Filho, Roberto Lent, and Suzana Herculano-Houzel. Equal numbers of neuronal and nonneuronal cells make the human brain an isometrically scaled-up primate brain. *Journal of Comparative Neurology*, 513(5):532–541, 2009.
- [4] Richard Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961.
- [5] N. A. Bernstein. *The Coordination and Regulation of Movements*. Pergamon Press, 1967.
- [6] F. Bosch. *Strength Training and Coordination: An Integrative Approach*. On Target Publications, 2015.
- [7] Frans Bosch. *Anatomy of Agility: movement analysis in sport*. 20/10 Publishers, 2020.
- [8] Avis H. Cohen and Peter Wallén. The neuronal correlate of locomotion in fish: “fictive swimming” induced in an in vitro preparation of the lamprey spinal cord. *Experimental Brain Research*, 41(1), 1980.
- [9] J. D. Cohen, G. Aston-Jones, and M. S. Gilzenrat. A theory of how the basal ganglia generates and uses neural signals. *Trends in Neurosciences*, 28(11):574–586, 2005.
- [10] Andrea d’Avella and Emilio Bizzi. Shared and specific muscle synergies in natural motor behaviors. *Proceedings of the National Academy of Sciences*, 102(8):3076–3081, 2005.
- [11] A. P. Georgopoulos, J. F. Kalaska, R. Caminiti, and J. T. Massey. On the relations between the direction of two-dimensional arm movements and cell discharge in primate motor cortex. *The Journal of Neuroscience*, 2(11):1527–1537, 1982.
- [12] Apostolos P. Georgopoulos, Andrew B. Schwartz, and Ronald E. Kettner. Neuronal population coding of movement direction. *Science*, 233(4771):1416–1419, 1986.
- [13] S. Grillner. Locomotion in vertebrates: central mechanisms and reflex interaction. *Physiological Reviews*, 55(2):247–304, 1975.
- [14] Hermann Haken, J. A. Scott Kelso, and H. Bunz. A theoretical model of phase transitions in human hand movements. *Biological Cybernetics*, 51(5):347–356, 1985.
- [15] Christopher M. Harris and Daniel M. Wolpert. Signal-dependent noise determines motor planning. *Nature*, 394(6695):780–784, 1998.

- [16] A. V. Hill. The heat of shortening and the dynamic constants of muscle. *Proceedings of the Royal Society B: Biological Sciences*, 126(843):136–195, 1938.
- [17] A. L. Hodgkin and A. F. Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of Physiology*, 117(4):500–544, 1952.
- [18] Neville Hogan. Adaptive control of mechanical impedance by coactivation of antagonist muscles. *IEEE Transactions on Automatic Control*, 29(8):681–690, 1984.
- [19] B. Hommel, J. Müsseler, G. Aschersleben, and W. Prinz. The theory of event coding (tec): a framework for perception and action planning. *Behavioral and Brain Sciences*, 24(5):849–878, 2001.
- [20] Vincent S. Huang, Adrian Haith, Pietro Mazzoni, and John W. Krakauer. Rethinking motor learning and savings in adaptation paradigms: model-free memory for successful actions combines with internal models. *Neuron*, 70(4):787–801, 2011.
- [21] Hiroshi Imamizu, Satoru Miyauchi, Tomoe Tamada, Yuka Sasaki, Ryousuke Takino, Benno Pütz, Toshinori Yoshioka, and Mitsuo Kawato. Human cerebellar activity reflecting an acquired internal model of a new tool. *Nature*, 403(6766):192–195, 2000.
- [22] W. James. *The Principles of Psychology*. Holt, Rinehart and Winston, 1890.
- [23] Eric R. Kandel, James H. Schwartz, Thomas M. Jessell, Steven A. Siegelbaum, and A. J. Hudspeth. *Principles of Neural Science*. McGraw-Hill Medical, New York, 5th edition, 2013.
- [24] M. Kawato, K. Furukawa, and R. Suzuki. A hierarchical neural-network model for control and learning of voluntary movement. *Biological Cybernetics*, 57(3):169–185, 1987.
- [25] John W. Krakauer, Maria-Felice Ghilardi, and Claude Ghez. Independent learning of internal models for kinematic and dynamic control of reaching. *Nature Neuroscience*, 2(11):1026–1031, 1999.
- [26] Jason J. Kutch and Francisco J. Valero-Cuevas. Challenges and new approaches to proving the existence of muscle synergies of neural origin. *PLoS Computational Biology*, 8(5):e1002434, 2012.
- [27] G. E. Loeb. Control implications of musculoskeletal mechanics. In *Proceedings of the 17th Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, volume 2, pages 1393–1394, Montreal, Canada, 1995.
- [28] Ian D. Loram and Martin Lakie. Direct measurement of human ankle stiffness during quiet standing: The intrinsic mechanical stiffness is insufficient for stability. *The Journal of Physiology*, 545(3):1041–1053, 2002.
- [29] D. Marr. A theory of cerebellar cortex. *Journal of Physiology*, 202(2):437–470, 1969.
- [30] K. Matsuoka. Sustained oscillations generated by mutually inhibiting neurons with adaptation. *Biological Cybernetics*, 52(6):367–376, 1985.

- [31] Matthew Millard, Thomas Uchida, Ajay Seth, and Scott L. Delp. Flexing computational muscle: Modeling and simulation of musculotendon dynamics. *Journal of Biomechanical Engineering*, 135(2):021005, 2013.
- [32] V. B. Mountcastle. The columnar organization of the neocortex. *Brain*, 120(4):701–722, 1997.
- [33] Karl M. Newell. Constraints on the development of coordination. In M. G. Wade and H. T. A. Whiting, editors, *Motor Development in Children: Aspects of Coordination and Control*, pages 341–360. Martinus Nijhoff, 1986.
- [34] Wilder Penfield and Edwin Boldrey. Somatic motor and sensory representation in the cerebral cortex of man as studied by electrical stimulation. *Brain*, 60(4):389–443, 1937.
- [35] J. P. Scholz and G. Schöner. The uncontrolled manifold concept: Identifying control variables for a functional task. *Experimental Brain Research*, 126:289–306, 1999.
- [36] W. Schultz, P. Dayan, and P. R. Montague. A neural substrate of prediction and reward. *Science*, 275(5307):1593–1599, 1997.
- [37] R. Shadmehr and F. A. Mussa-Ivaldi. Adaptive representation of dynamics during learning of a motor task. *Journal of Neuroscience*, 14(5):3208–3224, 1994.
- [38] Maurice A. Smith and Reza Shadmehr. Intact ability to learn internal models of arm dynamics in huntington’s disease but not cerebellar degeneration. *Journal of Neurophysiology*, 93(5):2809–2821, 2005.
- [39] Michele M. Solis and David J. Perkel. Rhythmic activity in a forebrain vocal control nucleus in vitro. *The Journal of Neuroscience*, 25(11):2811–2822, 2005.
- [40] Aparna Suvrathan, Hannah L. Payne, and Jennifer L. Raymond. Timing rules for synaptic plasticity matched to behavioral function. *Neuron*, 92(5):959–967, 2016.
- [41] E. Todorov and M. I. Jordan. Optimal feedback control as a theory of motor coordination. *Nature Neuroscience*, 5(11):1226–1235, 2002.
- [42] M. C. Tresch, V. C. K. Cheung, and A. d’Avella. Matrix factorization algorithms for the identification of muscle synergies. *Journal of Neurophysiology*, 95(4):2199–2212, 2006.
- [43] D. M. Wolpert and M. Kawato. Multiple paired forward and inverse models for motor control. *Neural Networks*, 11(7-8):1317–1329, 1998.